

Coursework report for Machine Learning

-

Module Code: CP60057E

Module Title: Machine Learning

Academic Year: 2025-2026

Assessment due date: 08/01/2026 23:59

-

Student Name: Mohammed Adnan Osman

Student ID: 33114153

Course Title: BSc Computer Science

Module Leader: Professor Massoud Zolgharni

Module Tutor: Eman Alajrami

-

Word Count: 2500 (Written Answers- Approx)

Table of Contents

Seminar Session-03: Clustering	3
Task 1	3
Seminar Week-04: Classification	5
Task 1	5
Task 2	8
Seminar Session-05: Random Forest.....	9
Task 1	9
Seminar Session-06: Perceptron	14
Task 1	14
Seminar Session-07: Support Vector Machines	17
Task 1	17
Task 2	21
Session Session-08: Neural Networks	22
Task 1	22
Task 2	22
Seminar Session-09: Image Classification	26
Task 1:	26
Seminar Session – 10: Convolutional Neural Networks	28
Task 1	28
Task 2	29
Seminar Session – 11: Regression (Linear Regression)	41
Task 1	41
Task 2	42
Task 3	43
Task 4	45
References	46

Seminar Session-03: Clustering

Task 1

- i. Use the Elbow Method to find the optimum value for k and show the results for clustering using the optimum k

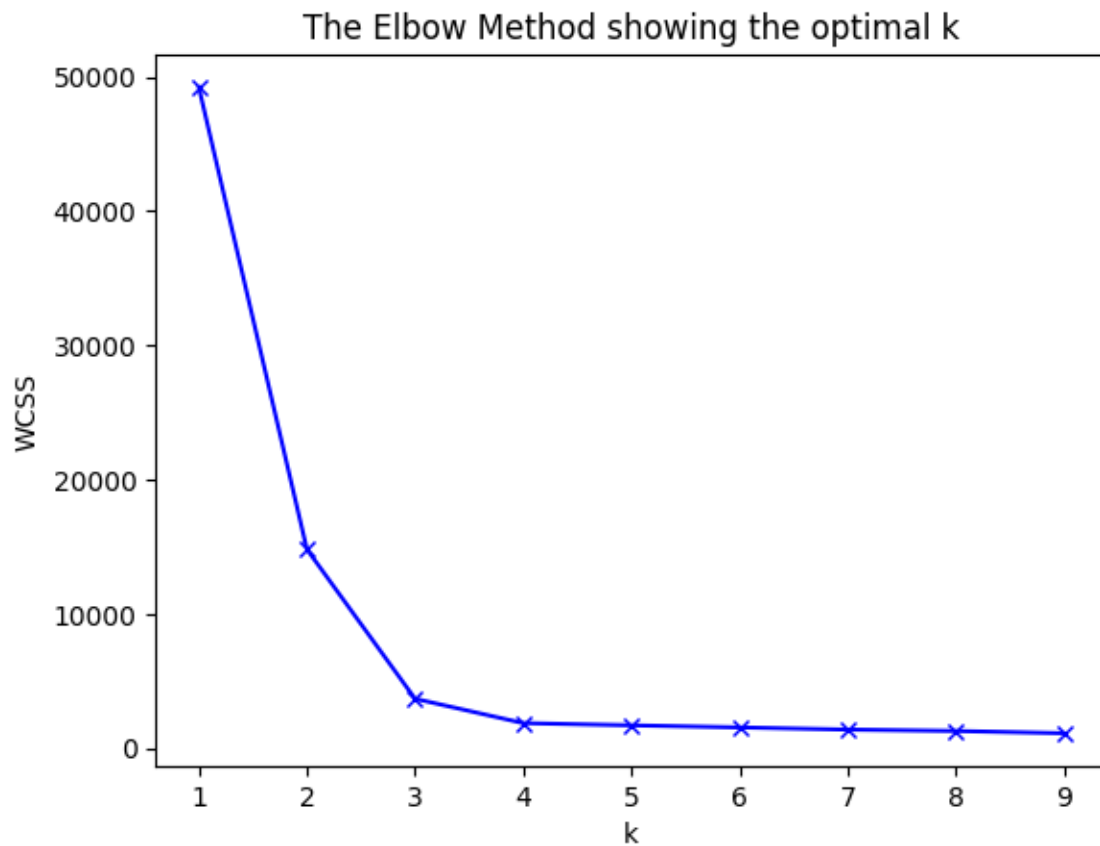


Figure 1: The Elbow Method Showing the Optimal Number of Clusters (K) - 3

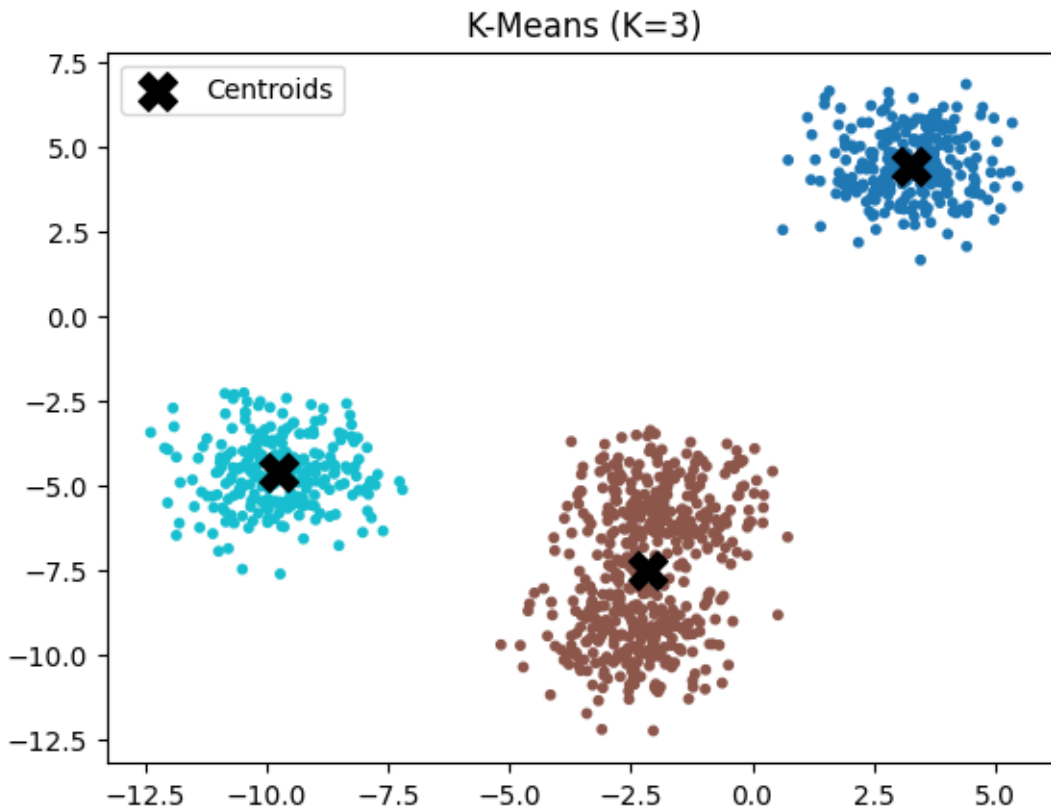


Figure 2: K-Means Clustering Result with $K = 3$

ii. Discuss your results

The Elbow Method was applied to determine the optimal number of clusters by analyzing the relationship between the number of clusters (K) and the within-cluster sum of squares (WCSS). The Elbow Method is appropriate here as it provides a clear visual trade-off between model complexity and clustering quality.

As shown in Figure 1, WCSS decreases sharply between $K = 1$ and $K = 3$, after which there are minimal improvements. This indicates diminishing returns beyond 3 clusters.

Using $K = 3$, the K-Means algorithm produced three well-separated clusters, as illustrated in Figure 2. Each cluster is centered around a clearly defined centroid with minimal overlap, suggesting that the dataset naturally separates into three meaningful groups in the feature space.

Although a small reduction in WCSS is observed at $K = 4$, this improvement does not justify the additional complexity. The Elbow Method relies on visual interpretation, other techniques could be used to further validate the chosen value of K .

However, K-means has inherent limitations. The algorithm assumes spherical clusters of similar size and is sensitive to initial centroid placement, which can lead to different results across runs. For datasets with non-spherical, irregular, or nested clusters, Density-Based Spatial Clustering would be more appropriate as it identifies clusters based on density rather than distance from centroids and does not require pre-specifying K. Additionally, the elbow method relies on visual interpretation; more rigorous approaches like the silhouette coefficient or gap statistic could provide quantitative validation of optimal cluster number.

Seminar Week-04: Classification

Task 1

- i. Try K-NN classifier for dataset provided in for different numbers of neighbors (K = 3 and K = 7). Also provide the relevant performance metrics such sensitivity, etc.

K = 3

Accuracy	0.867	
Precision	0.840	
Recall	0.907	
Specificity	0.827	
F1 Score	0.872	
AUC-ROC	0.928	
Confusion Matrix	124	26
	14	136

Figure 3: Results for k = 3

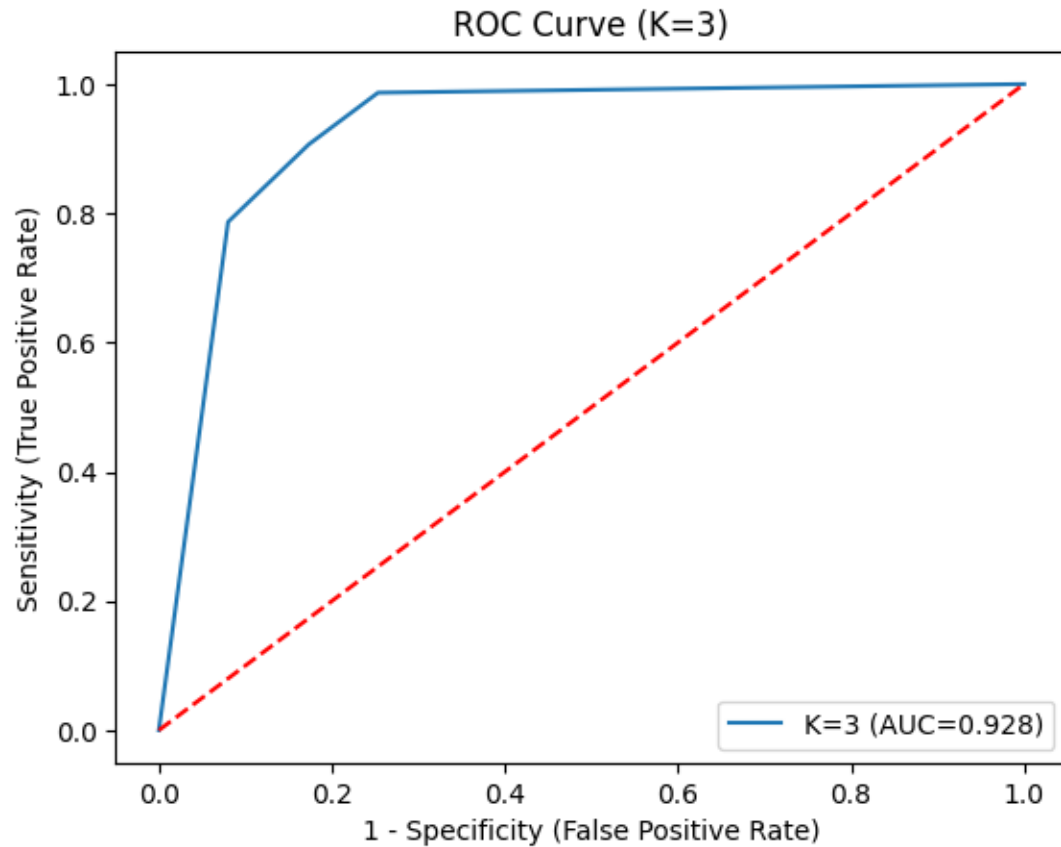


Figure 4 ROC Curve for K-NN Classifier with K = 3:

K = 7

Accuracy	0.877	
Precision	0.834	
Recall	0.940	
Specificity	0.813	
F1 Score	0.884	
AUC-ROC	0.943	
Confusion Matrix	122	28
	9	141

Figure 5: Results for K = 7

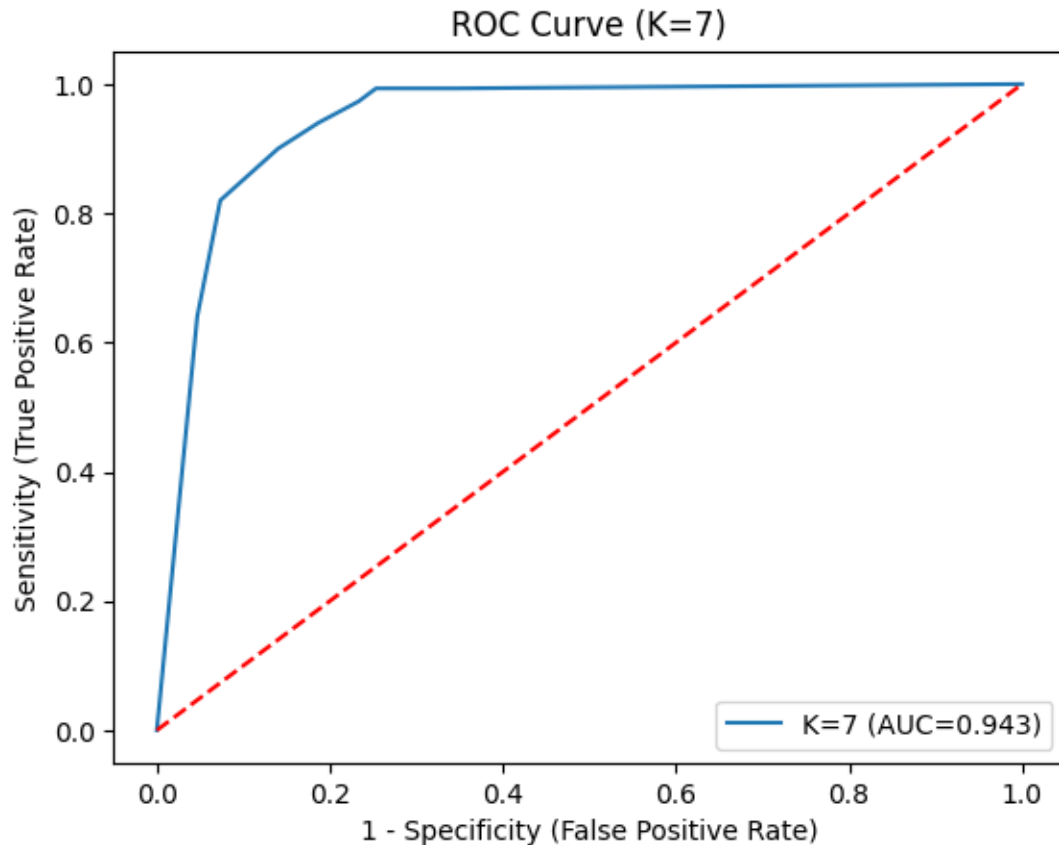


Figure 6: ROC Curve for K-NN Classifier with K = 7

ii. Which K provides better results? Justify your answer by using AUC of ROC curve.

The K-Nearest Neighbors classifier was tested using two cluster values: K = 3 and K = 7.

Table 1 Comparison of Performance Metrics for K-NN Classifier with K = 3 and K = 7.

Metric	K = 3	K = 7	Better Model
Accuracy	0.867	0.877	K = 7
Precision	0.840	0.834	K = 3
Recall (Sensitivity)	0.907	0.940	K = 7
Specificity	0.827	0.813	K = 3
F1 Score	0.872	0.884	K = 7
AUC (Area Under Curve)	0.928	0.943	K = 7

The k-Nearest Neighbors (k-NN) classifier was applied using K = 3 and K = 7 to evaluate the impact of neighborhood size on classification performance.

While both models achieved high accuracy, $K = 7$ demonstrated better overall performance in recall, F1 score, and AUC. The higher recall indicates improved detection of positive cases, while the increased F1 score reflects a better balance between precision and recall.

The AUC-ROC further supports this conclusion, as the model with $K = 7$ achieved a higher AUC value (0.943) compared to $K = 3$ (0.928). Since AUC evaluates class separability across all classification, it provides a more robust comparison than accuracy alone.

Although $K = 3$ achieved slightly higher precision and specificity, the overall trade-off favors $K = 7$, particularly when minimizing false negatives is important. However, increasing K may reduce sensitivity to patterns, and the optimal choice of K can vary depending on dataset characteristics and feature scaling.

Therefore, $K = 7$ was selected as the most appropriate value, offering improved generalization and balanced classification performance.

Task 2

- i. The script also returns to a value called 'f1_score'. Do some research to understand what this measure is.

The F1 score is a performance metric that combines precision and recall into a single value. It provides a balanced view of a classifier's ability to correctly identify positive cases while simultaneously minimizing both false positives and false negatives. So, the F1 score is useful when working with imbalanced datasets, where accuracy alone can be misleading.

$$F1 = [2 * (Precision * Recall)] / Precision + Recall$$

- ii. Repeat task-2 and this time justify your answer using f1_score.

Table 2 Comparison of Precision, Recall (Sensitivity), and F1 Score for K-NN Classifier with $K = 3$ and $K = 7$.

K	Precision	Recall (Sensitivity)	F1_Score
3	0.840	0.907	0.872
7	0.834	0.940	0.884

Using the F1 score to compare the two KNN models, the classifier with $K = 7$ achieved a higher F1 score (0.884) than the model with $K = 3$ (0.872), as shown in Table 2. This improvement is mainly due to the higher recall achieved by $K = 7$, indicating better detection of true positive cases while maintaining a similar level of precision.

- iii. Do you arrive at the same conclusion?

The same conclusion is reached when using both the F1 score and the AUC-ROC. In both cases, the $K = 7$ model demonstrates better overall performance and a stronger balance between sensitivity and precision, making it a better choice for this dataset.

The improved performance of $K=7$ over $K=3$ reflects the bias-variance tradeoff. Smaller K values ($K=3$) produce more flexible decision boundaries that closely fit training data but are sensitive to noise, resulting in higher variance. Larger K values create smoother boundaries through local averaging, reducing variance at the cost of slightly increased bias. The optimal $K=7$ strikes a balance, though cross-validation across a wider range would provide more robust selection.

However, K-NN has significant limitations. The algorithm has zero training time, but classification requires computing distances to all training points, resulting in $O(n \cdot d)$ computational complexity where n is training set size and d is dimensionality. This becomes too big for large datasets. Additionally, K-NN suffers from the curse of dimensionality when features increase, distances become less meaningful and performance degrades. For high-dimensional data, dimensionality reduction (PCA) or alternative methods like Random Forest or SVM would be more suitable. K-NN also treats all features equally so feature scaling is critical, and distance metric selection (Euclidean vs Manhattan) can significantly impact results.

Seminar Session-05: Random Forest

Task 1

- I. Try Random Forest Classifier for the Iris dataset example, and when creating decision trees from the bootstrapped dataset, use different number of features:
 - A subset of 2 features
 - A subset of 3 features
 - A subset of 4 features

Number of training samples: 118

Number of test samples: 32

Random Forest with 2 features per split:

Accuracy: 93.75 %

Confusion Matrix:

Table 1: Confusion Matrix – Random Forest Classifier (Iris Dataset)

Predicted:	<u>Setosa</u>	<u>Vericolour</u>	<u>Virginica</u>
Actual:			
<u>Setosa</u>	13	0	0
<u>Versicolour</u>	0	5	2
<u>virginica</u>	0	0	12

Feature Importance:

Table 2: Feature Importance – Random Forest Classifier (Iris Dataset)

Petal Length (cm)	0.446485
Petal Width (cm)	0.446313
Sepal Length (cm)	0.084740
Sepal Width (cm)	0.022461

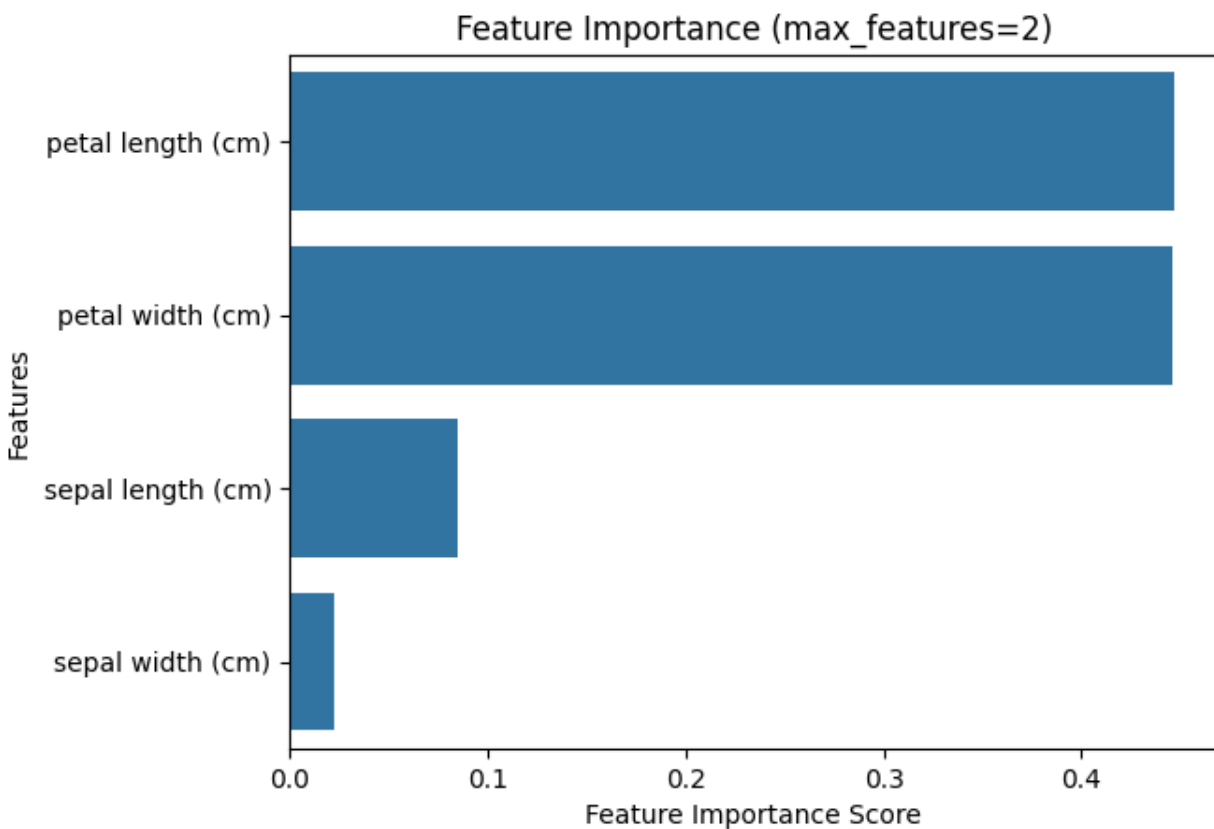


Figure 1: Feature Importance (max_features = 2)

Random Forest With 3 features per split:

Accuracy: 96.88 %

Confusion Matrix:

Predicted:	Setosa	Vericolour	Virginica
Actual:			
Setosa	13	0	0
Versicolour	0	6	1
Virginica	0	0	12

Figure 2: Confusion Matrix – Random Forest Classifier (max_features = 3)

Feature Importance:

Table 3: Feature Importance – Random Forest Classifier (max_features = 3)

Petal Length (cm)	0.540827
Petal Width (cm)	0.436527
Sepal Length (cm)	0.013891
Sepal Width (cm)	0.008755

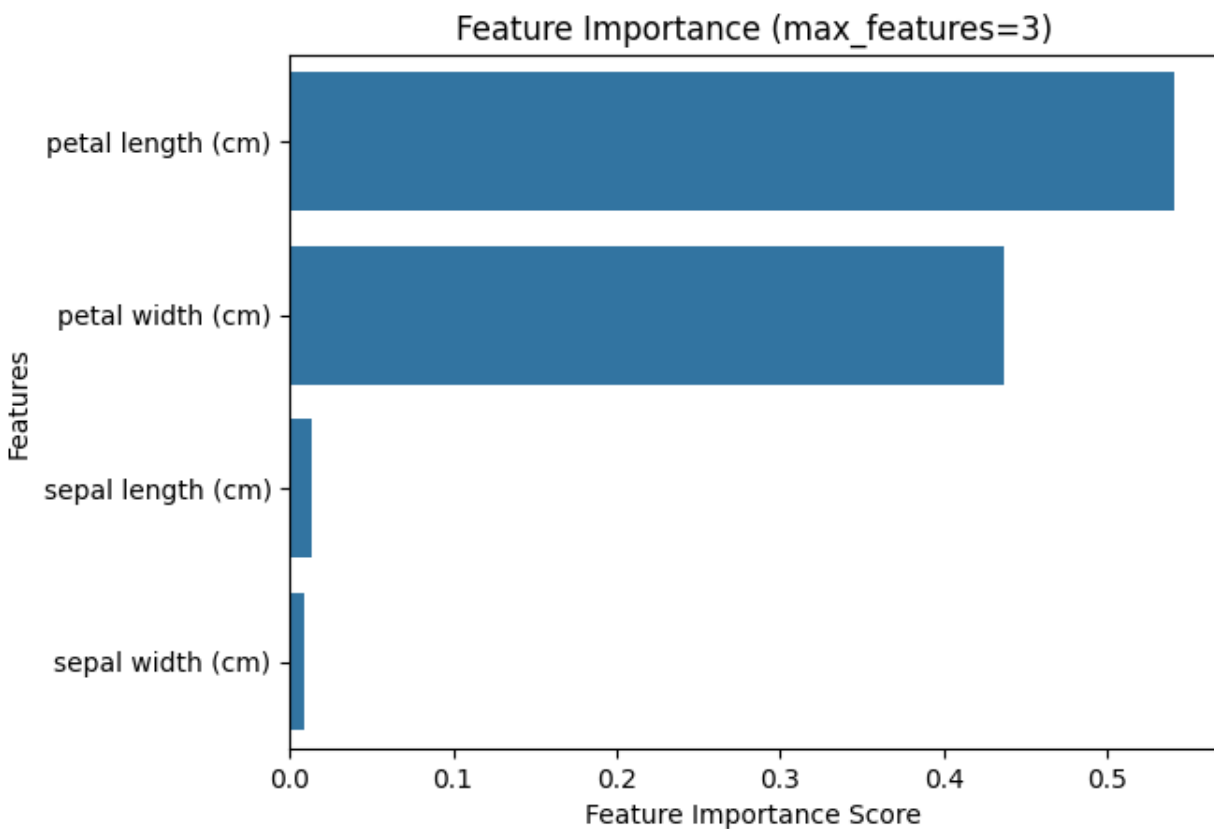


Figure 3: Feature Importance (max_features = 3)

Random Forest with 4 features per split

Accuracy: 93.75 %

Confusion Matrix:

Table 4: Confusion Matrix – Random Forest Classifier (max_features = 4)

Predicted:	Setosa	Versicolour	Virginica
Actual:			
Setosa	13	0	0
Versicolour	0	5	2
Virginica	0	0	12

Feature Importance:

Table 5: Feature Importance – Random Forest Classifier (max_features = 4)

Petal Length (cm)	0.535099
Petal Width (cm)	0.448016
Sepal Length (cm)	0.009444
Sepal Width (cm)	0.007442

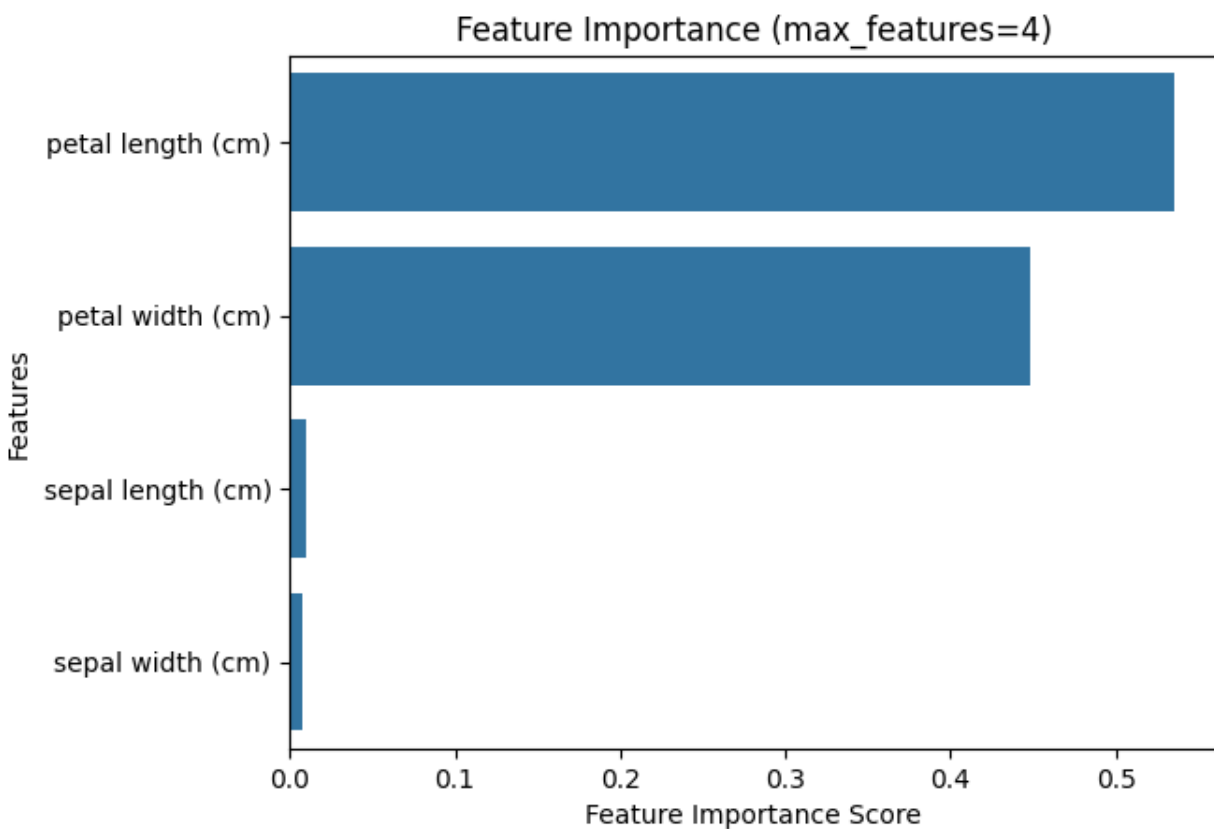


Figure 4: Feature Importance Visualization (max_features = 4)

- II. And compare the accuracy of your Random Forest Classifier. Which one is better?
Discuss your observations and justify your answer.

Results

Table 6: Comparison of Random Forest Performance with Different Numbers of Features per Split

Number Features per split	Accuracy (%)	Observations
2	93.75 %	Two versicolour samples were misclassified as virginica.
3	96.88 %	Only one of the versicolour samples were misclassified -> Thus the best accuracy
4	93.75 %	Same misclassifications as with 2 features

Table 6 compares the performance of the Random Forest classifier using different numbers of features per split. The model using 3 features per split achieved the highest accuracy of 96.88%, misclassifying only one Versicolor. So, this configuration generalized best to unseen test data.

When 2 features per split were used, the accuracy dropped to 93.75%, with two Versicolor misclassified as Virginica. Limiting the number of features reduces the information available to each decision tree, which weakened splitting decisions and reduced classification performance.

Using 4 features per split also resulted in an accuracy of 93.75%. In this case, all available features were used at every split, reducing the randomness of the Random Forest. This increased correlation between trees, limiting the benefit of ensemble learning and leading to lower performance than using 3 features.

Across all models, petal length and petal width were consistently identified as the most important features, while sepal measurements contributed very little to classification accuracy.

Overall, the Random Forest, with 3 features per split, provided the best trade-off between randomness and information gain, resulting in better generalization and the highest accuracy.

The Random Forest classifier outperformed a single decision tree due to ensemble learning through bagging, where multiple trees trained on different bootstrap samples reduced variance via majority voting. The use of the *max_features* parameter adds further

randomness by limiting the number of features considered at each split, which reduces correlation between trees. This explains why *max_features* = 3 achieved higher accuracy than using all four features, as fully shared feature sets reduce ensemble diversity.

Feature importance analysis showed that petal length and width contributed most to classification accuracy, while sepal features had minimal impact, indicating that petal measurements provide stronger species discrimination in the Iris dataset.

Despite its strong performance, Random Forest has limitations, including higher computational cost and reduced interpretability compared to simpler models. For applications requiring transparency, simpler classifiers may be preferable. For improved performance on more complex datasets, boosting-based methods such as Gradient Boosting or XGBoost could be explored as alternative approaches.

III. What is the largest number of features you may have?

The Iris dataset contains four input features: sepal length, sepal width, petal length, and petal width. So, the maximum number of features that can be used per split is 4. The number of features was controlled using the *max_features* parameter. While using all four features is possible, it removes randomness from the model, as every tree uses the same inputs at each split. Using fewer features increases tree diversity and typically improves generalization. Therefore, although the maximum is four, smaller values are often more effective when training.

Seminar Session-06: Perceptron

Task 1

- i. Apply the Perceptron algorithm to obtain a linear classifier and plot the results.

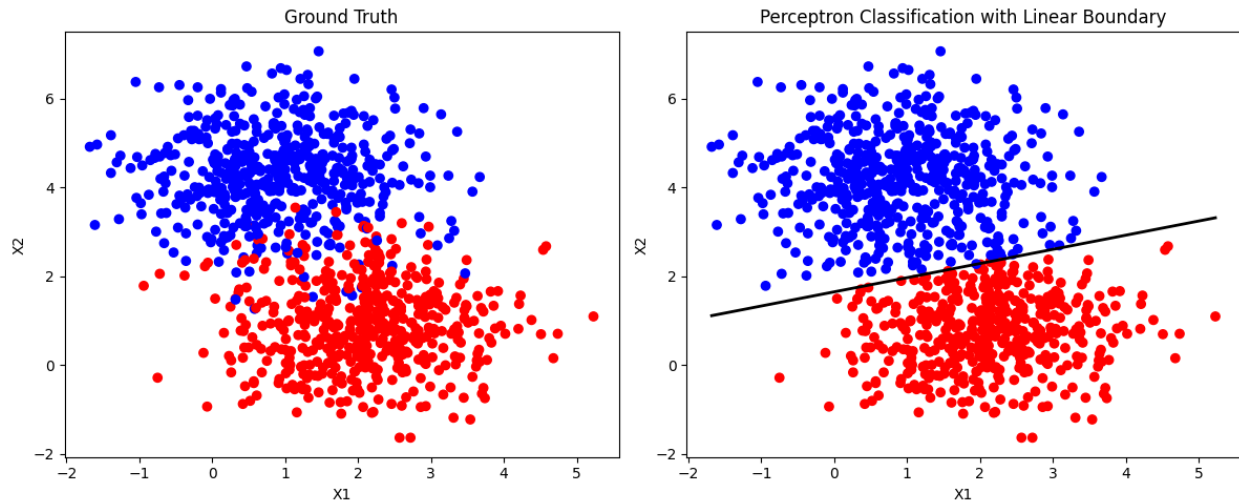


Figure 1: Perceptron Linear Classification Results

Final Weights:

W_0	25.000
W_1	4.850
W_2	-15.210

ii. Is the data linearly separable? Justify your answer.

The data is not linearly separable. The two classes overlap around the decision boundary, resulting in a non-zero training error. As the perceptron convergence theorem states that zero error is reached only for linearly separable data, which is not observed here.

iii. If linearly inseparable, what is the percentage of misclassified points using your linear classifier?

```
# Count misclassified points
misclassified = np.sum(y_hat != Y)
error_rate = (misclassified / len(Y)) * 100
```

Total points: 1000

Misclassified Points: 59

Error Rate: 5.90%

The error rate of 5.9% indicates that the dataset is not linearly separable, as a single linear decision boundary cannot perfectly separate the two classes. According to the Perceptron

Convergence Theorem, the perceptron algorithm only guarantees convergence to zero error when the data is linearly separable. The remaining misclassified points suggest overlapping class distributions near the decision boundary, which cannot be resolved by additional training iterations.

A key limitation of the perceptron is that it learns only linear decision boundaries and does not optimize the margin between classes. It finds a separating hyperplane without considering robustness or generalization. For datasets with overlapping classes, Support Vector Machines are more suitable, as they maximize the margin and are more resistant to noise. Alternatively, Multi-Layer Perceptron can learn non-linear decision boundaries through hidden layers, allowing them to model more complex class structures that a single-layer perceptron cannot represent.

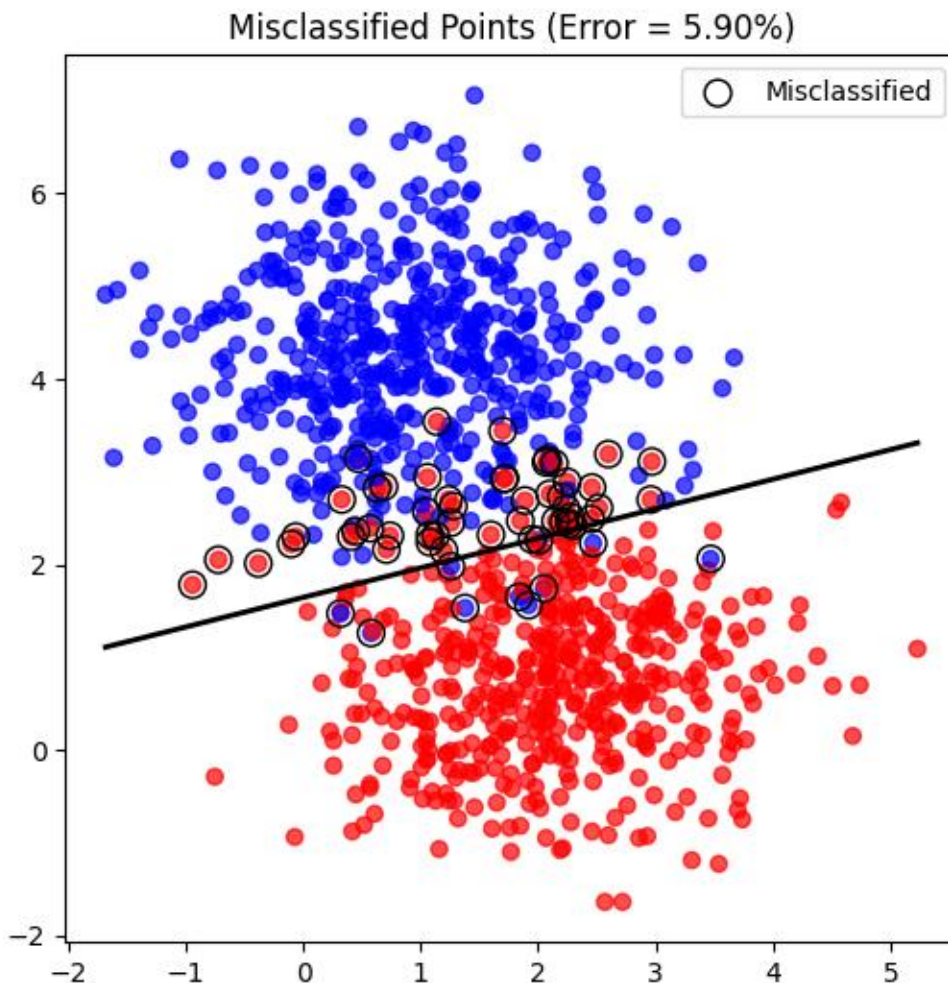


Figure 2: Misclassified Points (Error = 5.9%)

Seminar Session-07: Support Vector Machines

Task 1

For this task, download data_SVM.txt from Blackboard. This is a simple data set that contains 1000 entries with 2 features and corresponding class labels (0 or 1) for each entry.

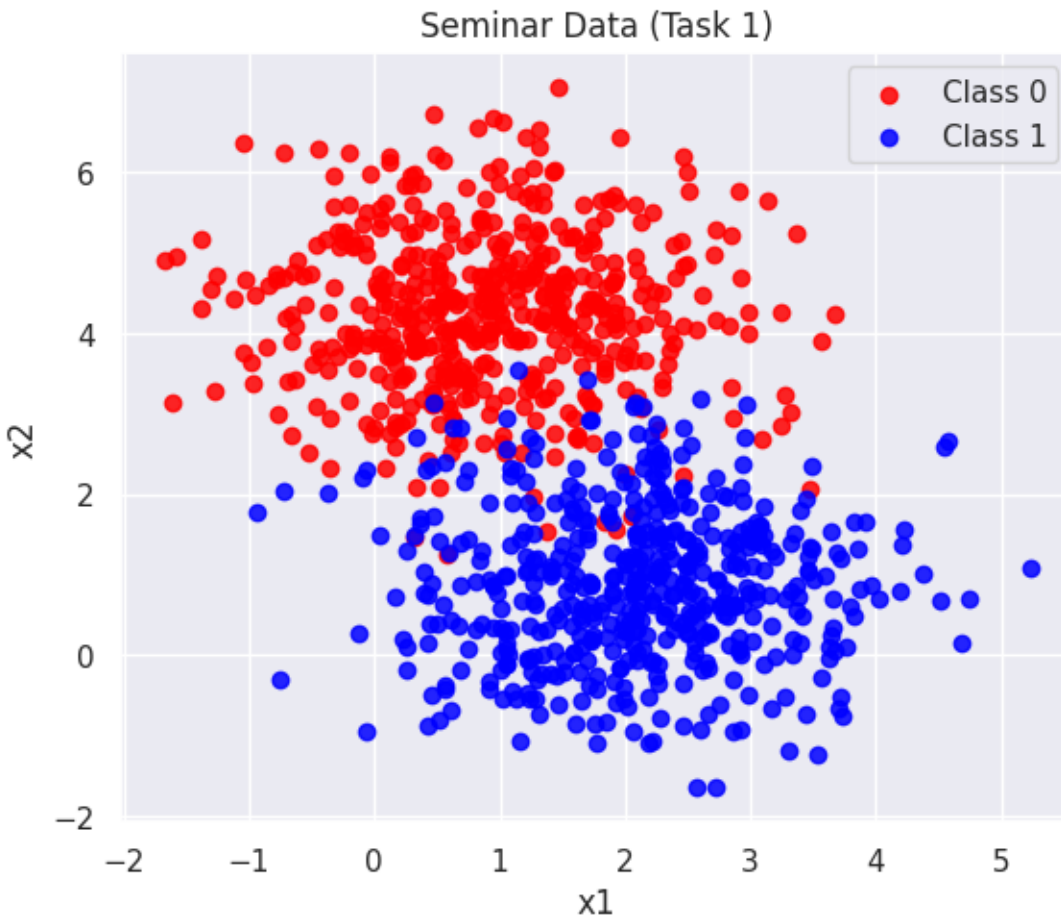


Figure 1: Visualization of the Seminar Dataset (Task 1)

- I. What is the percentage of misclassified points using your SVM classifier?

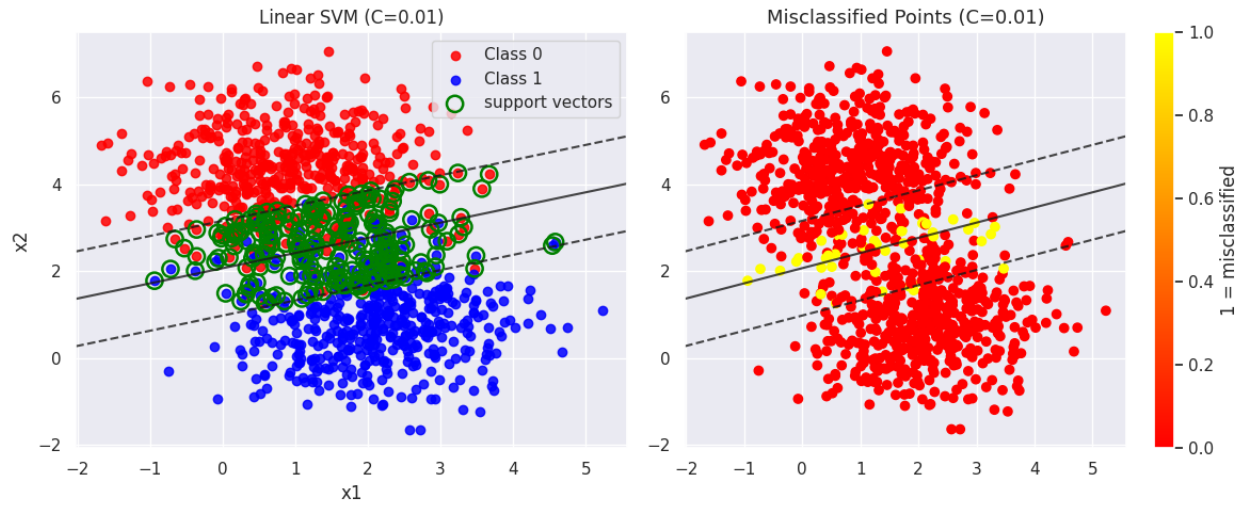


Figure 2: Linear SVM Classification with $C = 0.01$

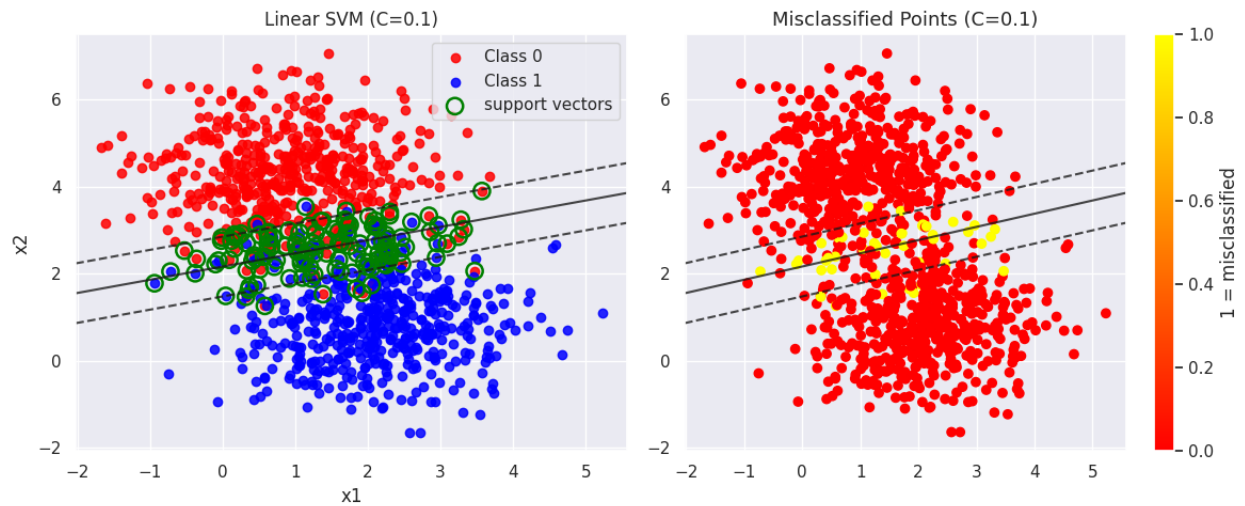
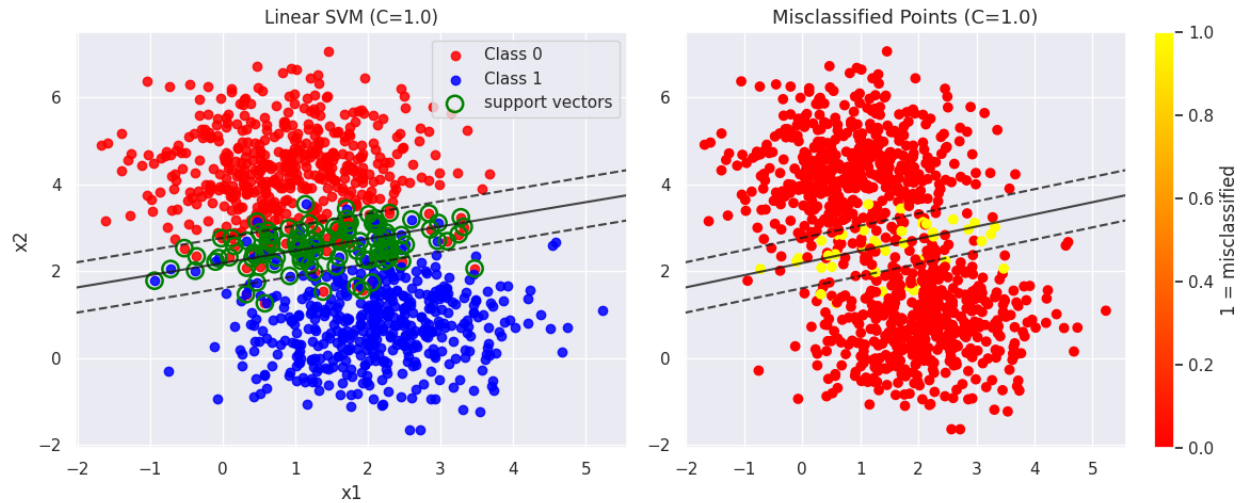


Figure 3: Linear SVM Classification with $C = 0.1$

Figure 4: Linear SVM Classification with $C = 1.0$ Table 1: SVM Performance for Different C Values

C Value	Accuracy (%)	Misclassified Points (%)
0.01	95.5%	4.5%
0.10	95.7%	4.3%
1.00	95.8%	4.2%

The percentage of misclassified points depends on the value of the regularization parameter C . As shown in Table 1, the lowest misclassification rate was achieved when $C = 1.00$.

- II. Try the algorithm for a range of different values of C parameter [0.01, 0.1, 1.0], and provide the results together with your understanding of the influence of the C parameter.

Table 2: Effect of the Regularization Parameter (C) on SVM Performance Metrics

C Value	Accuracy (%)	Misclassified Points (%)	Support Vectors (Total)	Margin Width ($2/ W $)
0.01	95.5%	4.5%	224	2.0595
0.10	95.7%	4.3%	131	1.3149
1.00	95.8%	4.2%	109	1.1158

The results show that increasing C slightly improves accuracy while reducing the number of misclassified points, support vectors, and margin width.

Parameter C controls the trade-off between maximizing the margin and minimizing classification errors. A low C value (0.01) allows more misclassifications, resulting in a

wider margin and a larger number of support vectors. This improves robustness and generalization but slightly reduces accuracy.

A higher C value (1.0) penalizes misclassifications more strongly, producing a narrower margin with fewer support vectors and higher accuracy. However, very large values of C can reduce generalization and increase the risk of overfitting.

Overall, increasing C makes the SVM stricter about classification errors, improving accuracy at the cost of reduced margin width and generalization.

III. Compare your SVM results with those from the Perceptron in week-6.

Table 3: Comparison Between the Perceptron (Week 6) and Support Vector Machine (Week 7) Classifiers

Algorithm	Dataset Size	Accuracy (%)	Misclassified / Error Rate (%)	Reason?
Perception (Week 6)	1000	94.1	5.9	Found a decision boundary, no explicit margin, more sensitive to noise
SVM (Week 7)	1000	95.5 – 95.8	4.2 – 4.5	Found the best hyperplane, fewer errors, more stable.

Both the Perceptron and Support Vector Machine are linear classifiers that aim to separate data using a hyperplane. However, the SVM extends the perceptron approach by maximizing the margin, which improves stability and generalization, particularly for overlapping or noisy data.

As shown in Table 3, the SVM achieved higher accuracy (95.5 - 95.8%) and a lower misclassification rate (4.2 - 4.5%) compared to the Perceptron (94.1% accuracy, 5.9% error). The perception focuses only on separating the classes and does not explicitly control the margin, making it more sensitive to noise.

In contrast, the SVM selects the optimal hyperplane by balancing margin width and classification error through the regularization parameter C. This results in fewer errors and more robust performance, explaining why the SVM outperformed the perception on the same dataset.

The superior performance of the Support Vector Machine compared to the Perceptron (95.8% vs 94.1%) is explained by the principle of margin maximization, which is grounded

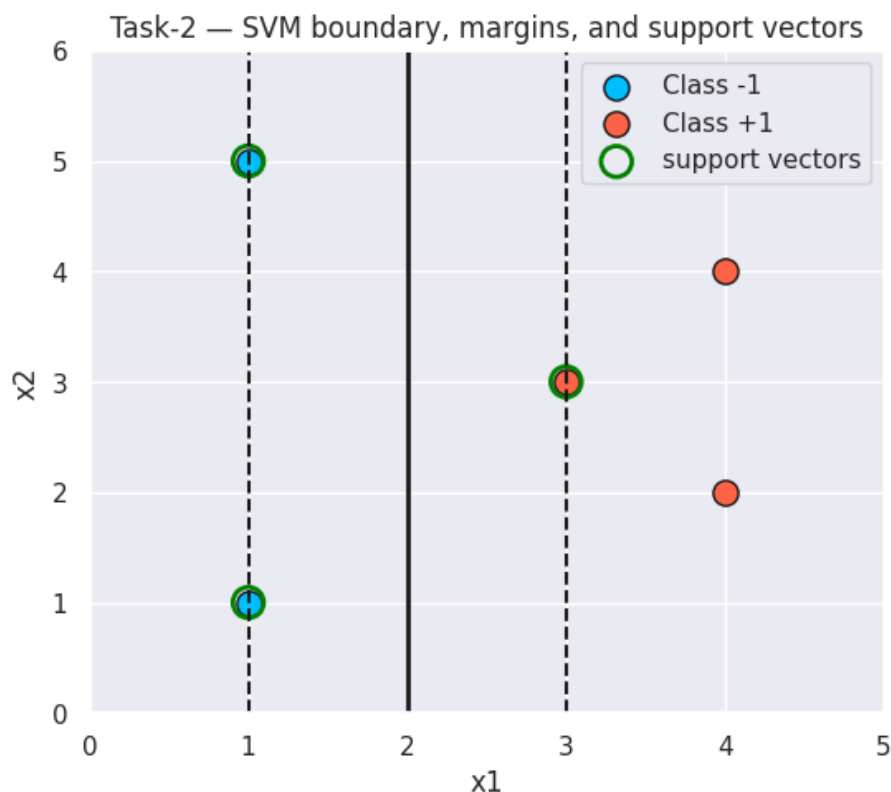
in structural risk minimization. By maximizing the margin between classes, SVM reduces sensitivity to noise and improves generalization to unseen data. Unlike the Perceptron, only a subset of training points (support vectors) defines the decision boundary, making the model more stable.

Here, a linear kernel was used. However, SVMs can be extended using the kernel trick to model non-linear decision boundaries without explicit feature transformations. Despite its advantages, SVM has limitations, including higher computational cost and sensitivity to hyperparameters such as C , which requires careful tuning. For very large datasets or multi-class problems, alternatives such as Random Forests or Neural Networks may be more practical.

Task 2

Consider we are learning a linear SVM for a training dataset shown in a table below, described with two features x_1 and x_2 , and binary target y .

- I. Draw the decision boundary of the trained SVM and identify the support vectors.



The vertical line which separates the two classes is where $X_1 = 2$.

Support Vectors

Negative Class:

This is when $Y = -1$ so $= (1,1)$ and $(1,5)$

Positive Class:

This is when $Y = +1$ so $(3,3)$

Margin Width

Width = $2 / ||W|| = 2$

II. Calculate the parameters of the trained SVM (i.e., w_1 , w_2 , b).

$W = [1, 0]$

$B = -2$

Session Session-08: Neural Networks

Task 1

- i. Explain how a Multi-Layer Perceptron can address the limitation of a single-layer Perceptron.

A single-layer perception can only form a single linear decision boundary and therefore can solve only linearly separable problems. As a result, it fails on non-linear patterns such as XOR, clusters, or circular class boundaries.

A Multi-Layer Perceptron overcomes this limitation by introducing one or more hidden layers. Each hidden neuron applies a nonlinear activation to a weighted input, allowing the network to learn intermediate feature representations. When combined across layers, these transformations produce nonlinear, curved, or piecewise-linear decision boundaries.

By progressively reshaping the input space, an MLP can separate complex patterns that are not linearly separable. So, MLPs are significantly more expressive than single layer perception and can learn complex decision functions.

Task 2

- i. Try a few different configurations for the network architecture and discuss your observations in the logbook.

A small network (2) with one hidden layer and two neurons achieved a test accuracy of 31.58% and predicted all samples as a single class. This indicates severe underfitting, as the network lacked sufficient capacity to learn the nonlinear structure of the dataset.

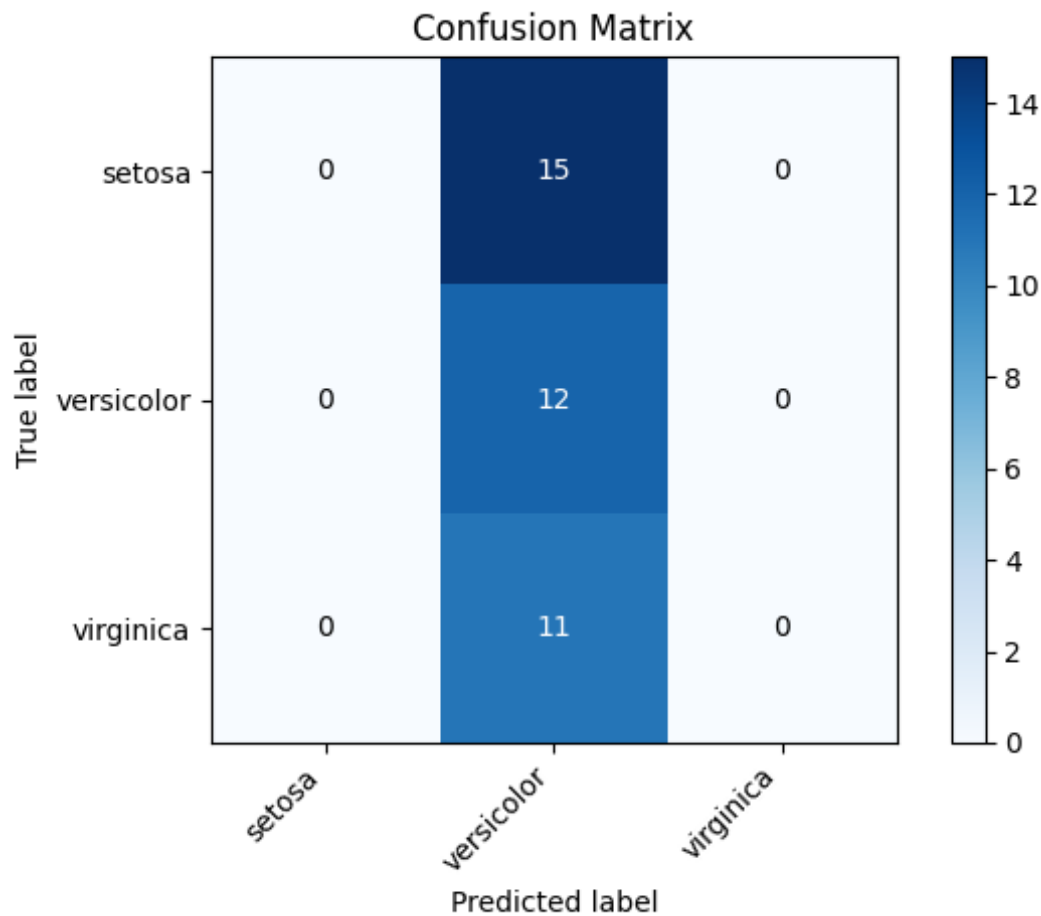


Figure 5: SMALL network (underfitting) — (2,)

A medium-sized network (4,4,2) achieved a test accuracy of 97.37%, misclassifying only one sample. This configuration provided a good balance between model capacity and generalization, demonstrating that moderate depth and width are sufficient for accurate classification of the Iris dataset.

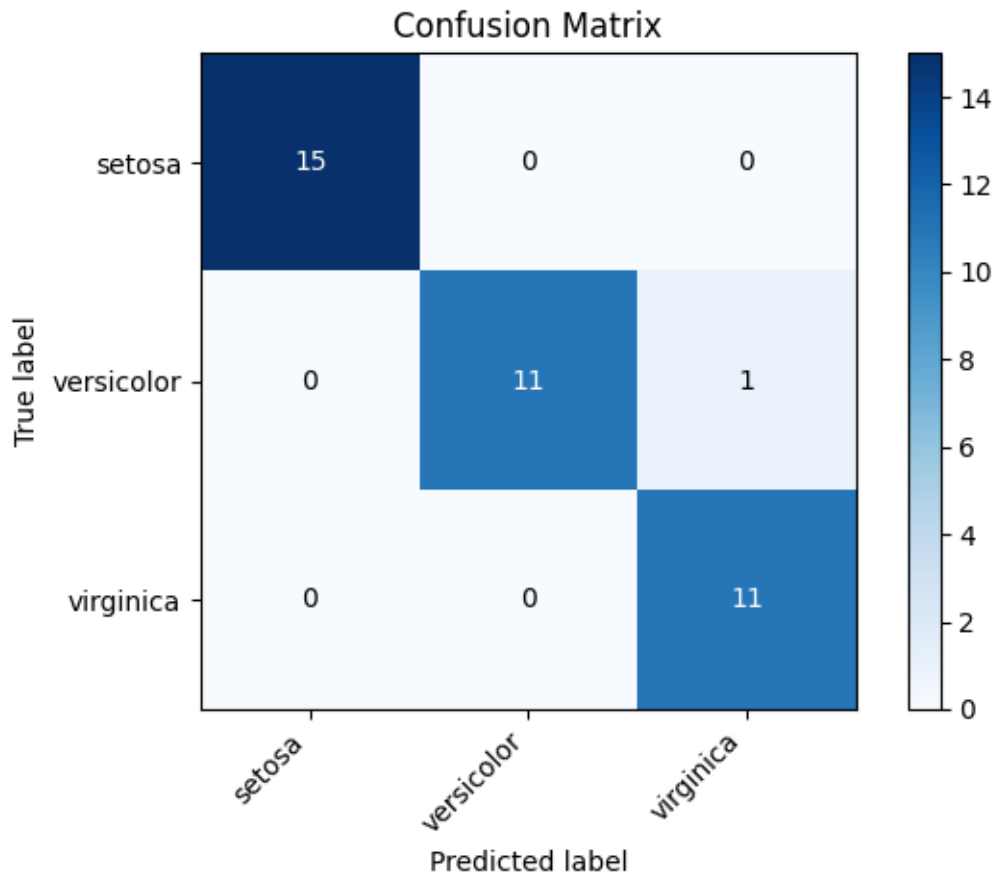


Figure 6: MEDIUM network (good fit) , (4,4,2)

A larger network (20,10,5) also achieved 97.37% accuracy, with no improvement over the medium network. This shows that increasing the number of neurons and layers does not necessarily improve performance on simple datasets and may only increase computational cost or risk overfitting.

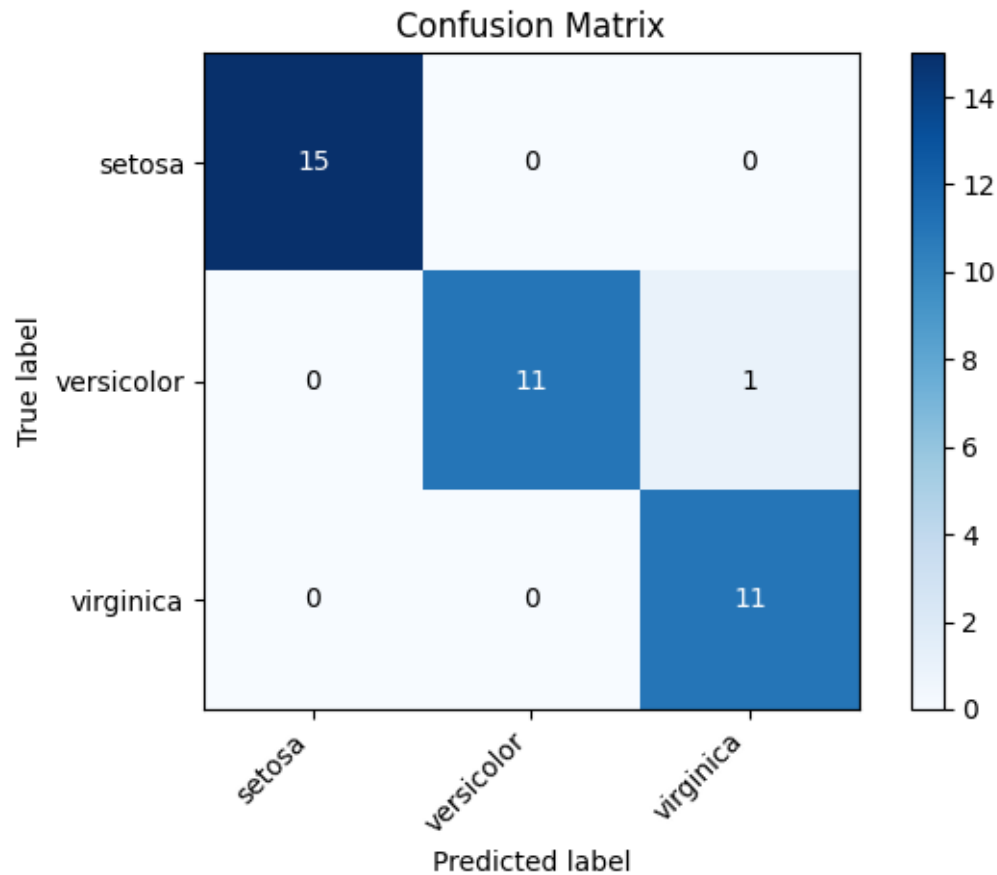


Figure 7: LARGE network (higher capacity) , (20,10,5)

Overall, the results show that network capacity must be matched to data complexity, with moderate architectures often providing the best generalization.

- ii. How does the number of neurons and hidden layers affect the performance of the network?

The number of neurons and hidden layers directly affect model capacity. Networks with too few neurons (2) suffer from underfitting, as they cannot capture the data's complexity and patterns, resulting in very low accuracy (31.58%).

A moderate-sized network (4,4,2) achieved the best performance (97.37%), demonstrating strong generalization and sufficient capacity to model the dataset. Increasing network size further (20,10,5) did not improve accuracy, as the Iris dataset is simple. Larger networks may only increase training costs and risk overfitting on more complex data.

Overall, moderate architecture provides the best balance between capacity and generalization.

Multi-Layer Perceptron overcome the limitations of single layer perceptron's by introducing hidden layers with non-linear activation functions. While the perception in Week 6 could only learn linear decision boundaries, MLPs can model non-linear relationships by applying successive transformations to the input space. Training is achieved using the backpropagation algorithm, which computes gradients through the network and updates weights using gradient descent, allowing the model to learn complex patterns.

In more complex datasets, larger networks would require regularization techniques such as dropout, L2 regularization, or early stopping to prevent overfitting. Additionally, MLPs act as black-box models; therefore, for problems requiring interpretability, simpler models may be preferred. For image-based tasks, convolutional neural networks are more suitable

iii. Can you find any architecture that yields 100% accuracy on test set?

None of the tested architectures achieved 100% test accuracy. The highest accuracy obtained was 97.37%, achieved by both the (4,4,2) and (20,10,5) networks. Although the Iris dataset is well-separated, I couldn't get it using these numbers. The results were, however, only one sample away from perfect classification.

Seminar Session-09: Image Classification

Task 1:

i. Choose the best value for K in K-NN. Justify your choice.

The best value of $K = 11$, selected using 5-fold cross-validation. Among the tested values, $K = 11$ achieved the highest mean validation accuracy with relatively low variance. Smaller values of K are overfitted and are sensitive to noise, while larger values over smooth the decision boundary. Therefore, $K = 11$ provides the best bias-variance.

```
def compute_distances_no_loops(self, X):
    X2 = np.sum(X**2, axis=1, keepdims=True)
    Y2 = np.sum(self.X_train**2, axis=1)[None, :]
    XY = X @ self.X_train.T
    d2 = X2 + Y2 - 2 * XY
    np.maximum(d2, 0, out=d2)
    return np.sqrt(d2)
```

A full no-loop distance computation was implemented to calculate the Euclidean distances for k-NN classification.

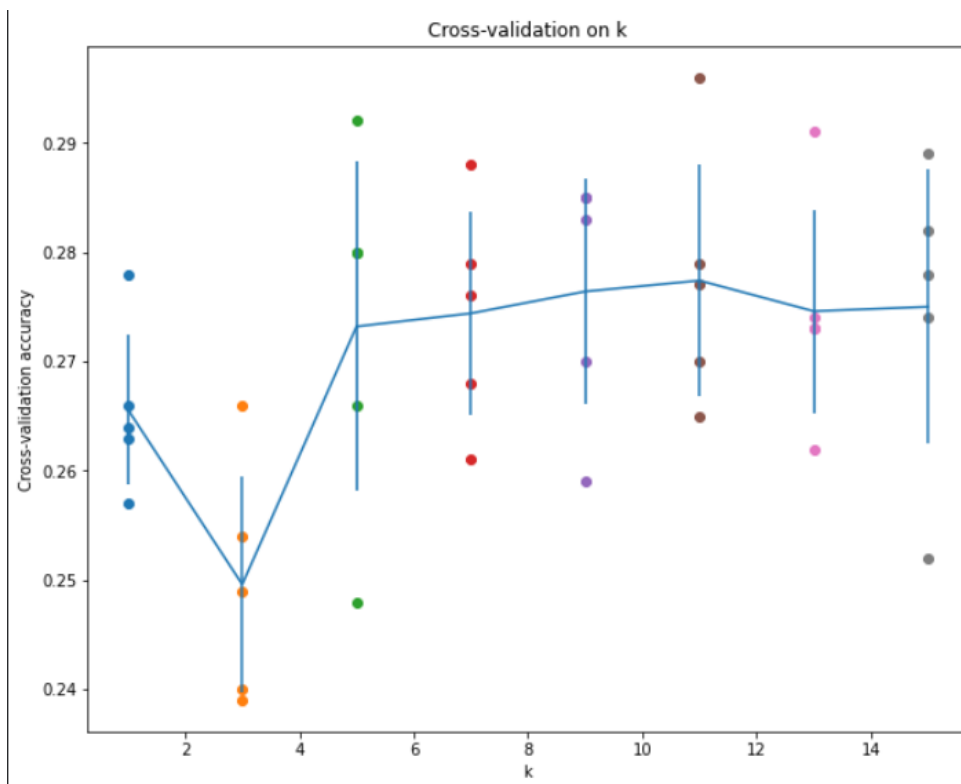


Figure 8: Cross Validation on K

ii. What is the level of accuracy?

Using $K = 11$, the KNN classifier achieved an accuracy of 32.4%, correctly classifying 3240 out of 10,000 images in the CIFAR-10 test set. This performance is expected for KNN to be applied to raw pixel data on a complex image dataset and highlights the limitations of KNN without learned feature representations.

The poor performance of the K-NN classifier on CIFAR-10 illustrates the curse of dimensionality in practice. Each image is represented by 3,072 raw pixel features ($32 \times 32 \times 3$), and in such high-dimensional spaces, Euclidean distance becomes less meaningful as data points tend to appear similarly distant from one another. Additionally, raw pixel values encode low-level brightness information rather than semantic structure; small spatial shifts or lighting changes can significantly alter distances despite the image content remaining the same. As a result, distance-based methods such as K-NN struggle to generalize on complex image data. Improved performance would require meaningful feature representations, either through feature extraction techniques like HOG or learned representations using convolutional neural networks

Seminar Session – 10: Convolutional Neural Networks

Task 1

- I. Provide the details of your calculations

Source Pixels:

[3 0 1]

[2 6 2]

[2 4 1]

Filter:

[-1 0 1]

[-2 0 2]

[-1 0 1]

Multiply:

$$[(3 * -1) + (0 * 0) + (1 * 1)] = (-3) + (0) + (+1) = -2$$

$$[(2 * -2) + (6 * 0) + (2 * 2)] = (-4) + (0) + (4) = 0$$

$$[(2 * -1) + (4 * 0) + (1 * 1)] = (-2) + (0) + (1) = -1$$

Add rows:

$$-2 + 0 + (-1) = -3$$

Answer: - 3

Task 2

- i. Train your CNN model for a few more iterations. Try it for 10, 20, 50 epochs.

```
acc_10_k33 = train_and_report(epochs=10, kernel=(3,3))  
acc_20_k33 = train_and_report(epochs=20, kernel=(3,3))  
acc_50_k33 = train_and_report(epochs=50, kernel=(3,3))
```

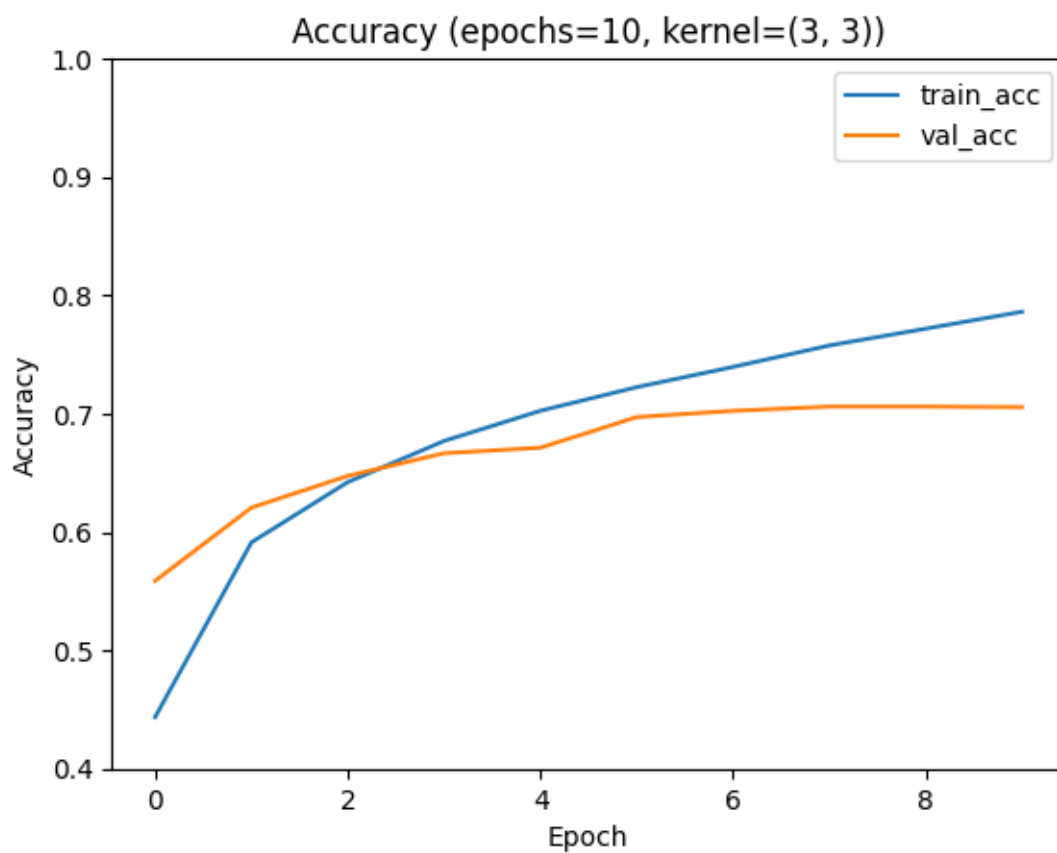


Figure 9: (Accuracy – 10 epochs, kernel 3×3)

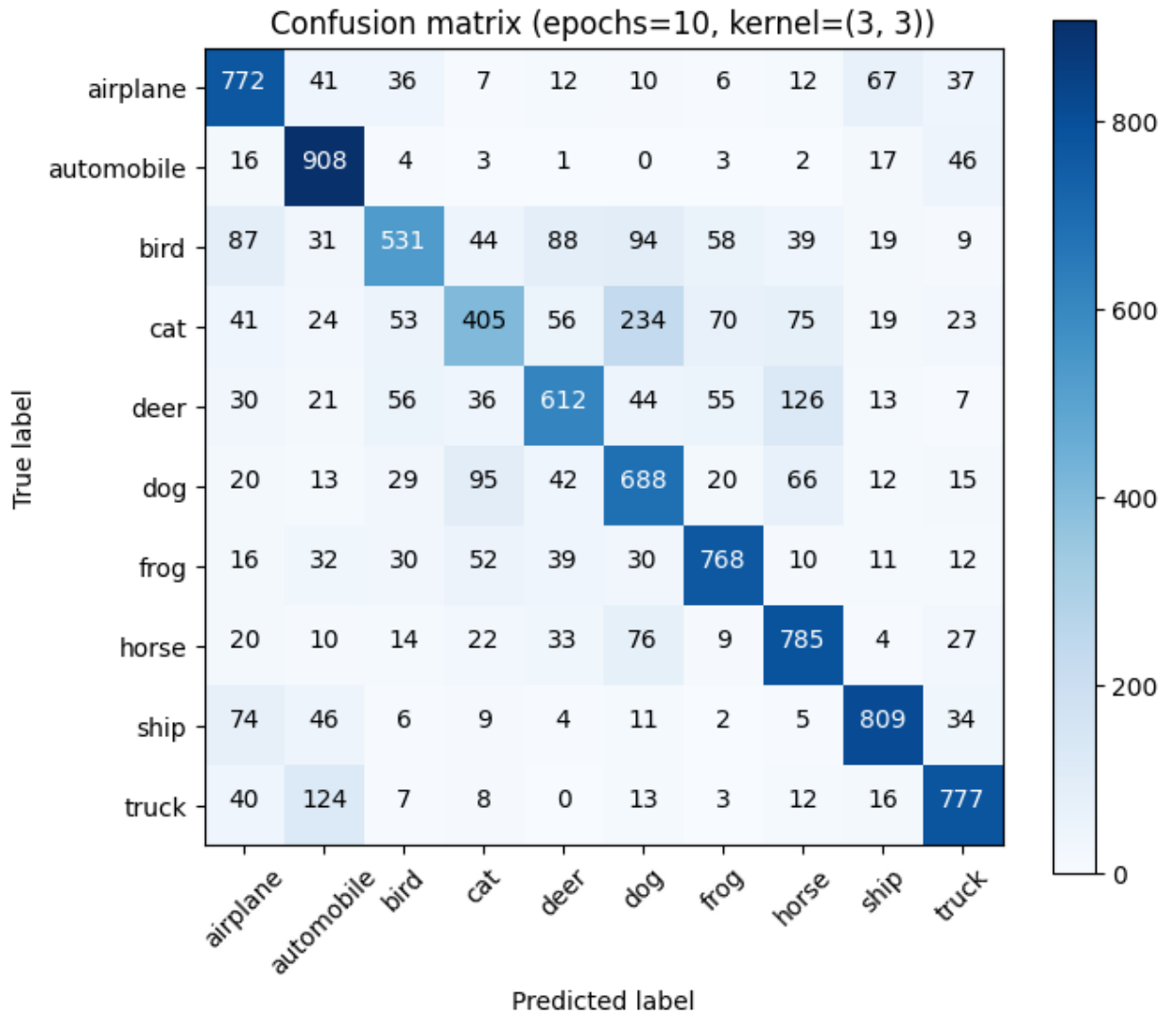


Figure 10: (Confusion Matrix – 10 epochs, kernel 3×3)

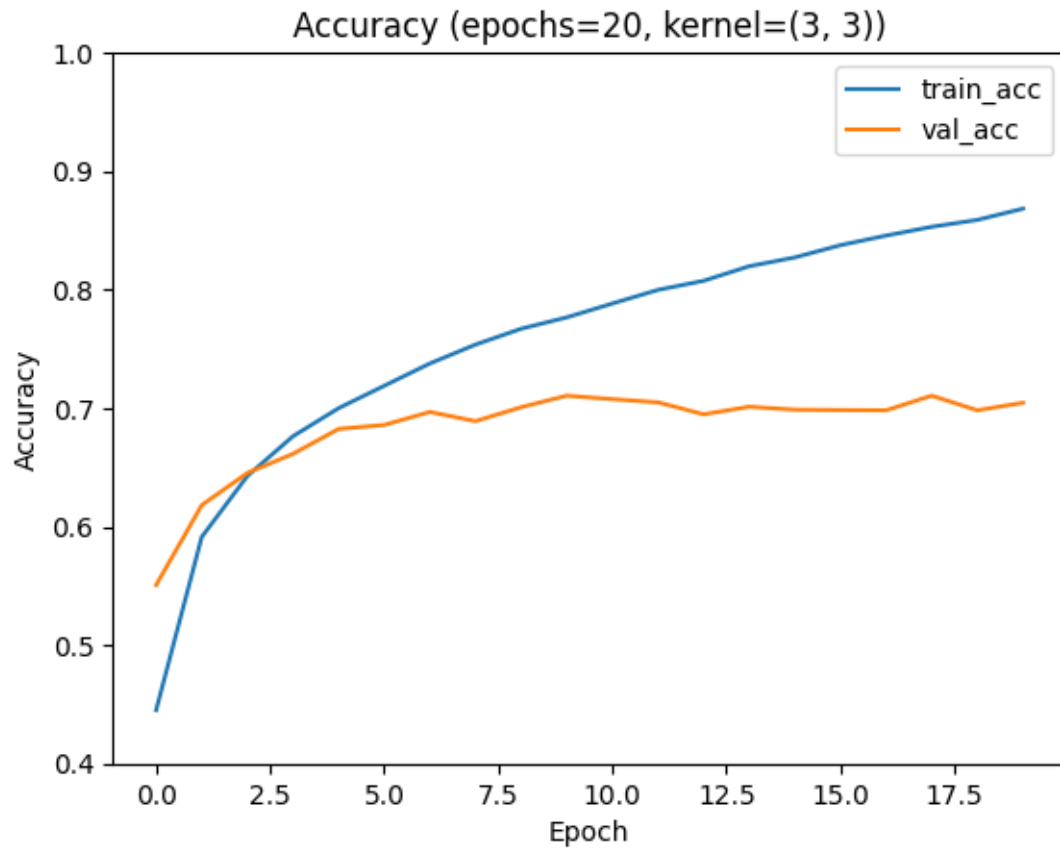


Figure 11: (Accuracy – 20 epochs, kernel 3×3)

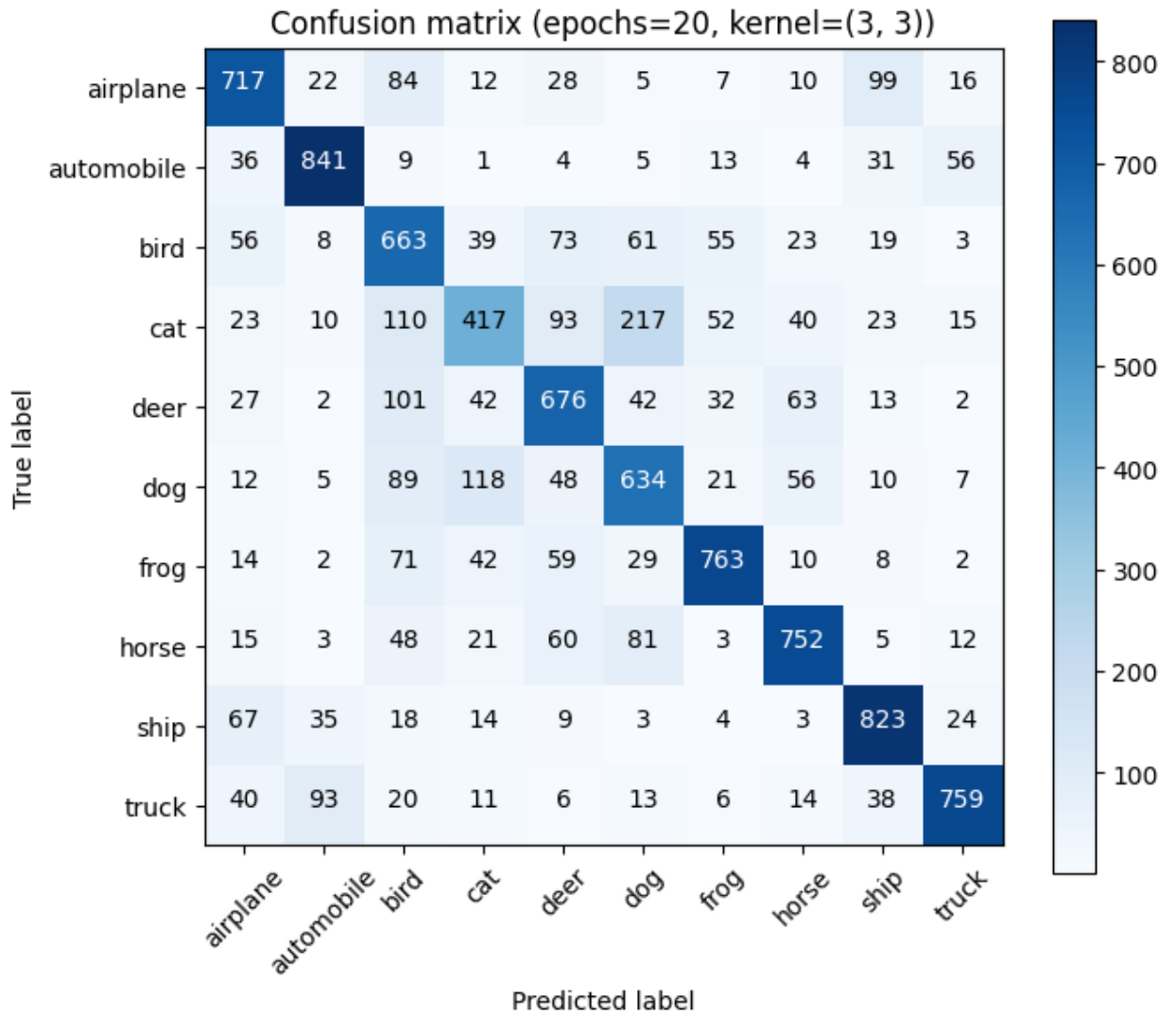


Figure 12: (Confusion Matrix – 20 epochs, kernel 3×3)

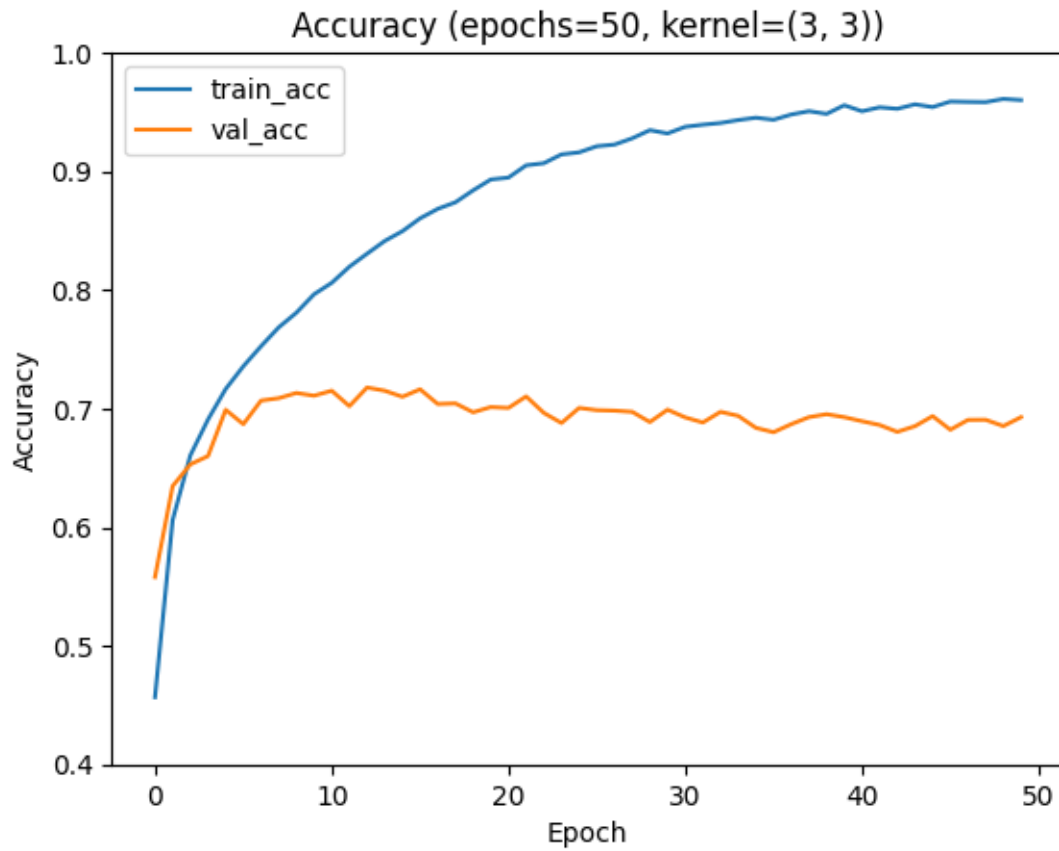


Figure 13: (Accuracy – 50 epochs, kernel 3×3)

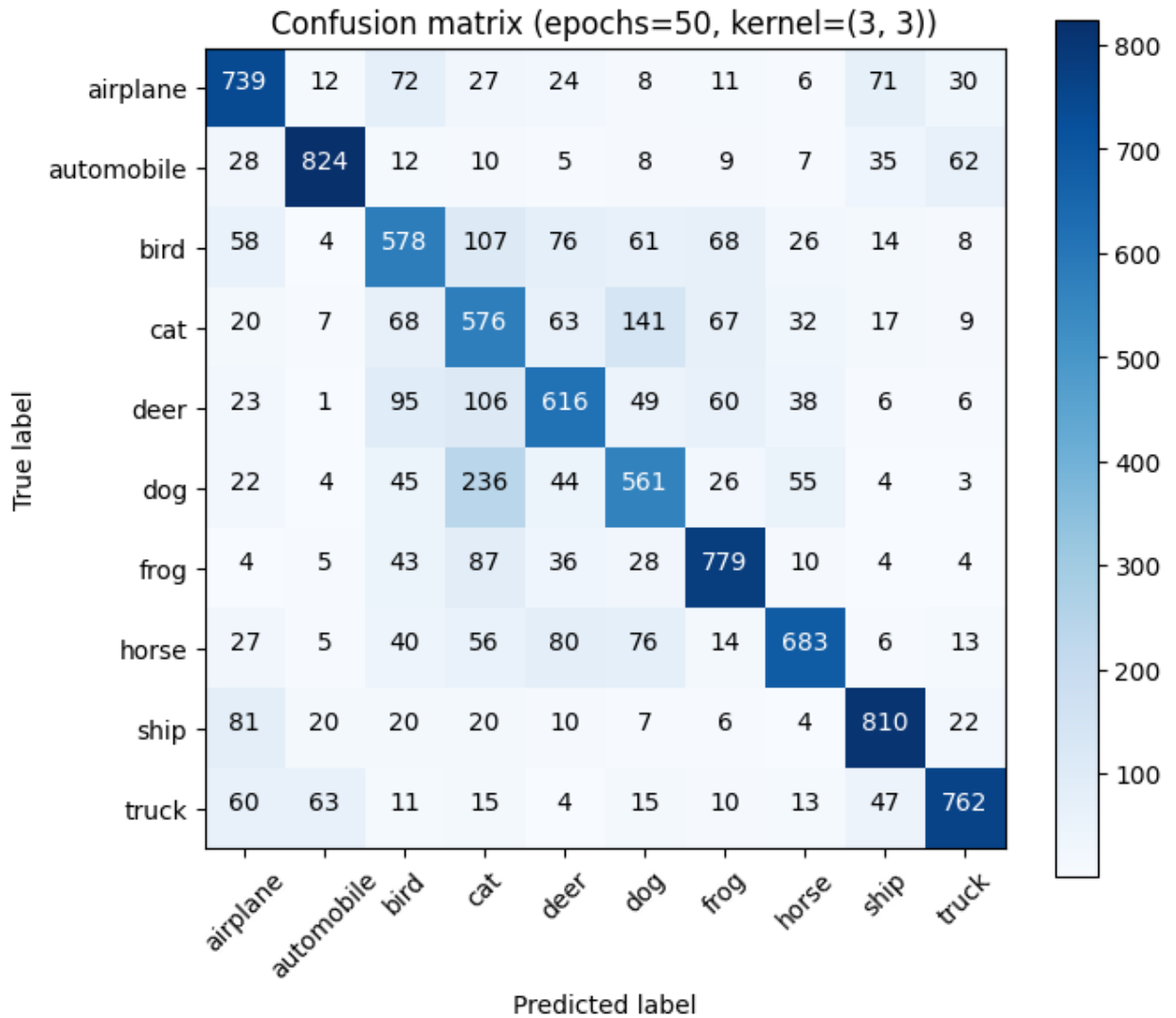


Figure 14: (Confusion Matrix – 50 epochs, kernel 3×3)

- ii. How does the accuracy change? Discuss your observations.

Test accuracy increases slightly from 0.6974 (10 epochs) to 0.6992 (20 epochs), indicating initial learning improvement. When training is extended to 50 epochs, accuracy decreases to 0.6928, suggesting overfitting. Overall, accuracy improves marginally at first and then stabilizes or slightly declines with excessive training.

Kernel Size	Epochs	Test Accuracy
3 X 3	10	0.6974
3 X 3	20	0.6992
3 X 3	50	0.6928

- iii. If accuracy is improving, explain why. If accuracy is getting worse, why do you think that is?

Accuracy improves from 10 to 20 epochs because CNN has more iterations to learn meaningful features, reducing the loss and improving generalization. When training reaches 50 epochs, accuracy decreases due to overfitting, where the model memorizes training patterns rather than learning features that generalize well to unseen data.

- iv. Change the kernel size from (3, 3) to (5, 5) in your CNN model.

```
acc_10_k55 = train_and_report(epochs=10, kernel=(5,5))  
acc_20_k55 = train_and_report(epochs=20, kernel=(5,5))  
acc_50_k55 = train_and_report(epochs=50, kernel=(5,5))
```

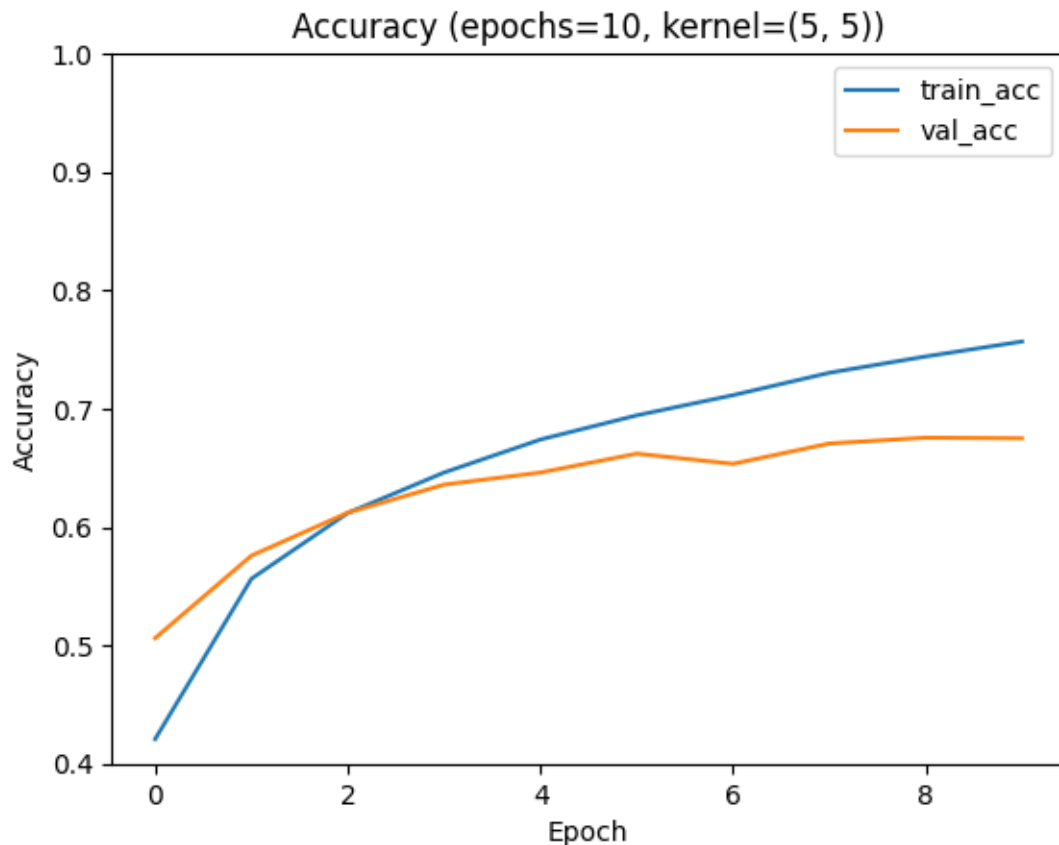


Figure 15: (Accuracy – 10 epochs, kernel 5×5)

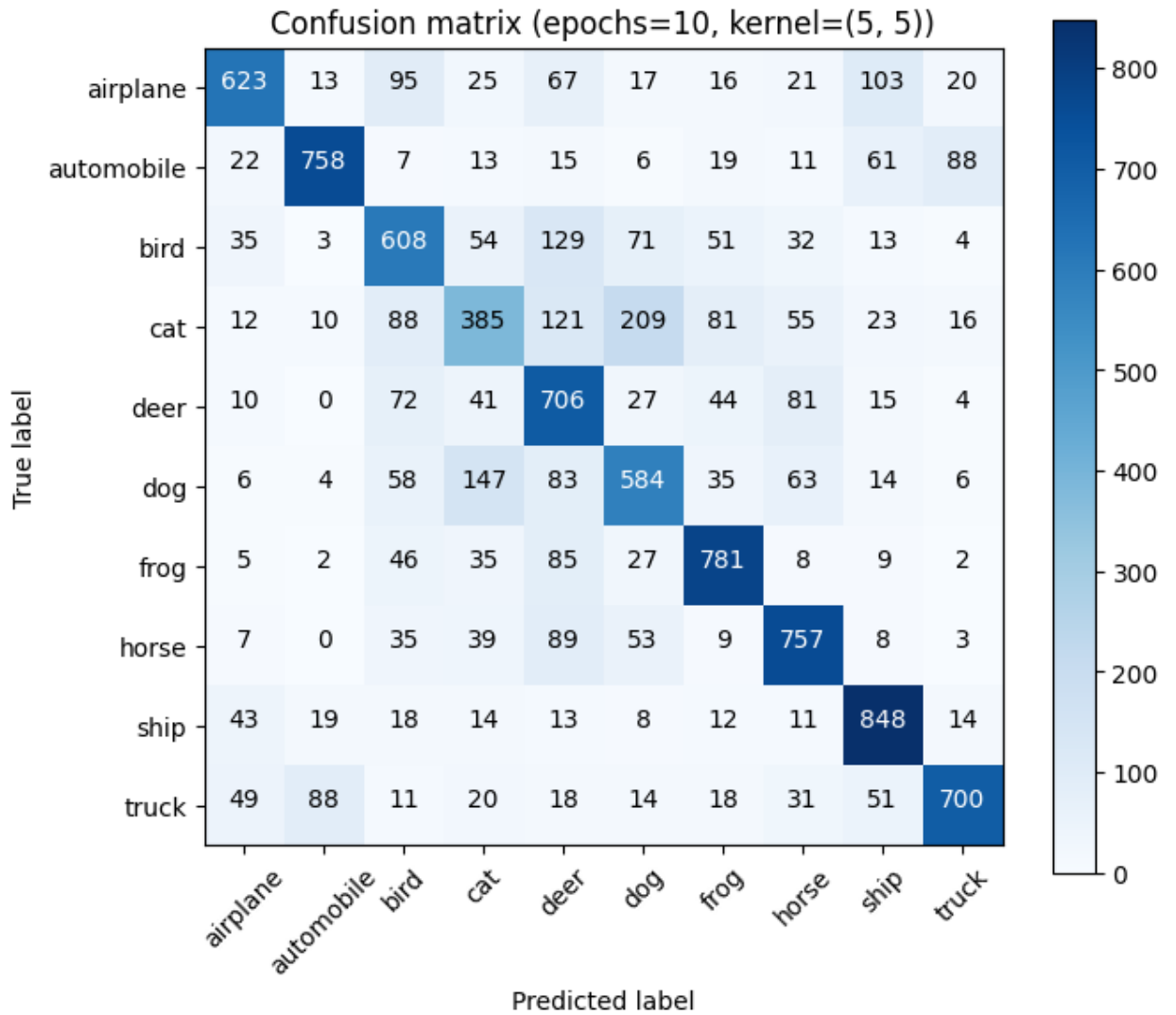


Figure 16: (Confusion Matrix – 10 epochs, kernel 5×5)

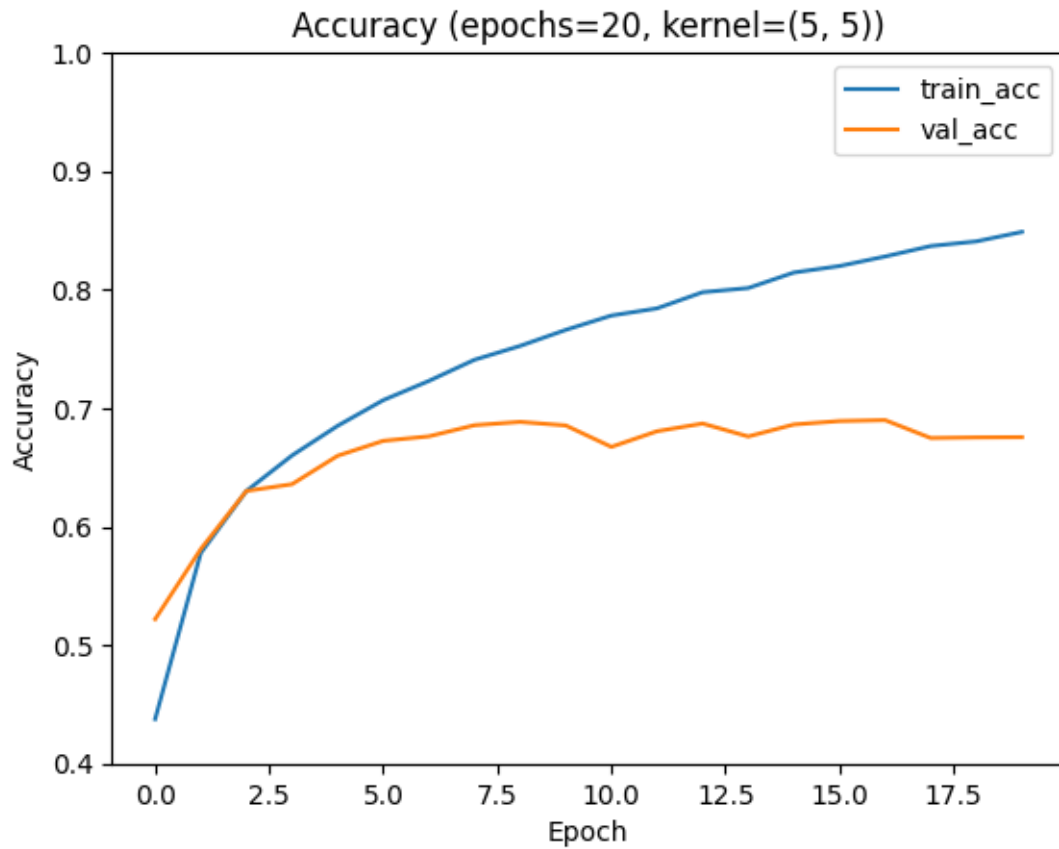


Figure 17: (Accuracy – 20 epochs, kernel 5×5)

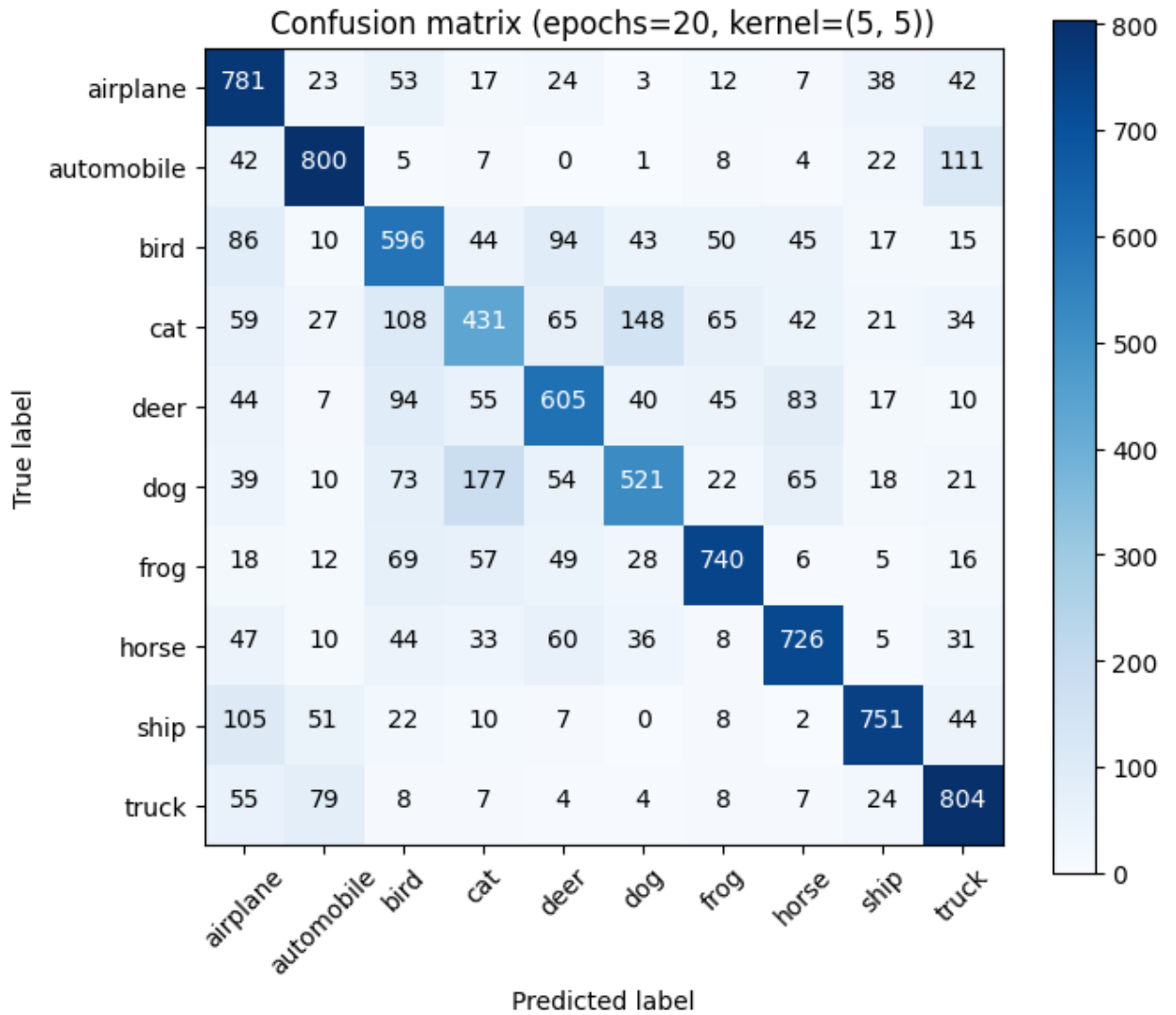


Figure 18: (Confusion Matrix – 20 epochs, kernel 5×5)

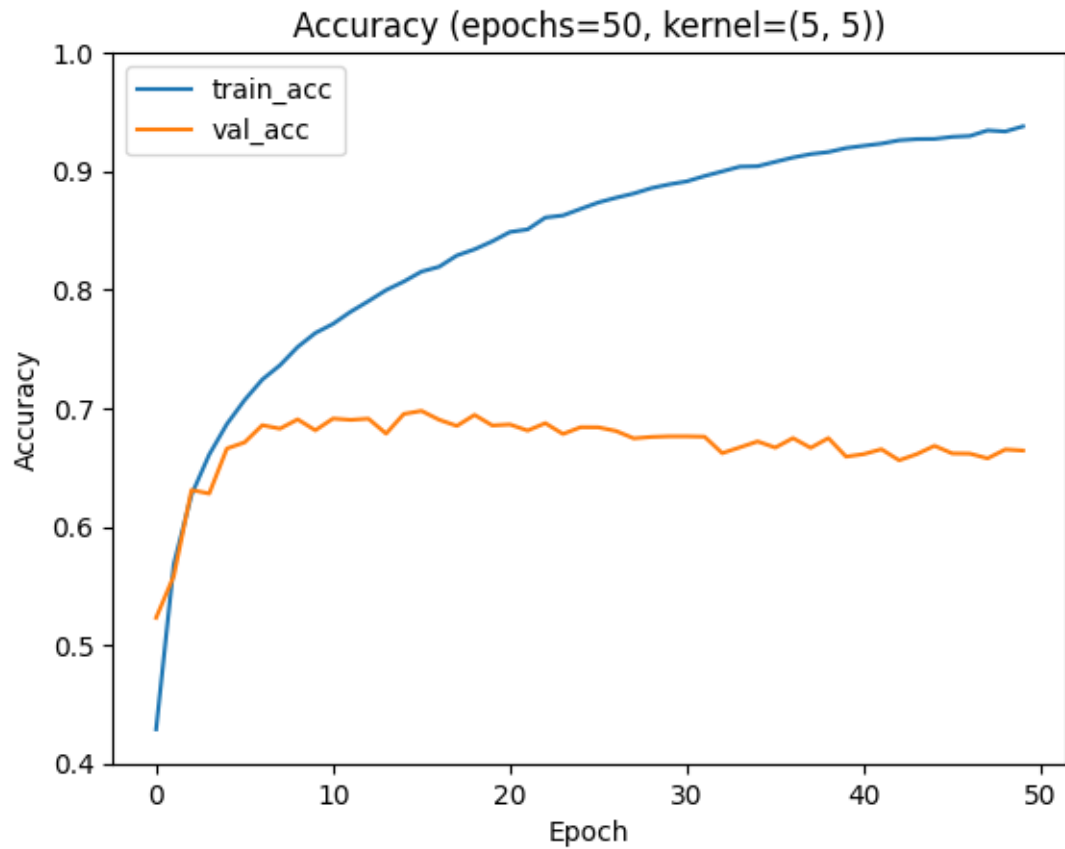


Figure 19: (Accuracy – 50 epochs, kernel 5×5)

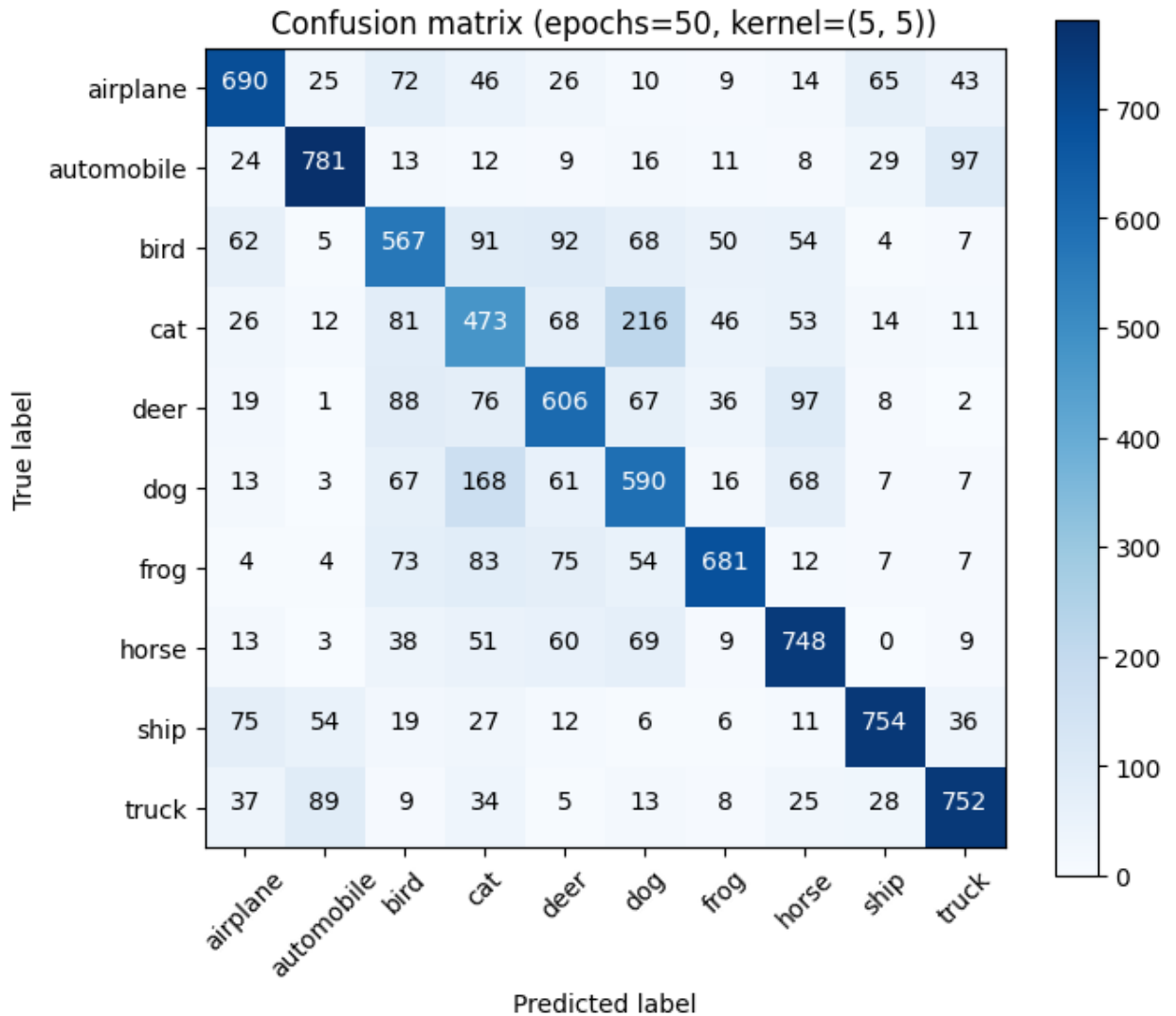


Figure 20: (Confusion Matrix – 50 epochs, kernel 5×5)

iv. Would that help to improve the accuracy?

Kernel Size	Epochs	Test Accuracy
5 X 5	10	0.6750
5 X 5	20	0.6755
5 X 5	50	0.6642

No. Using a 5×5 kernel resulted in lower test accuracy across all epochs. Larger kernels increase parameter count and computational complexity, making the model more prone to overfitting on small images such as CIFAR-10. The 3×3 kernel therefore provides better performance and efficiency.

Unlike K-NN, which relies on Euclidean distance between raw pixel values, CNNs exploit the spatial structure of images through local connectivity and weight sharing.

Convolutional layers learn hierarchical features, progressing from low-level edges to higher-level shapes and object parts, enabling the model to capture semantic information rather than pixel-level similarity. This explains why CNNs are far more effective on complex image datasets such as CIFAR-10.

The results also demonstrate key training and architectural considerations. Test accuracy peaked at 20 epochs and declined at 50 epochs, indicating overfitting, where the model begins to memorize training patterns rather than generalize to unseen data. Techniques such as early stopping, data augmentation, dropout, and batch normalization would mitigate this issue. Additionally, 3×3 kernels consistently outperformed 5×5 kernels, as smaller kernels require fewer parameters while preserving effective receptive fields when stacked.

Seminar Session – 11: Regression (Linear Regression)

Task 1

- I. Calculate the Least Squares Error for this model.

$$Y = 6.49x + 30.18$$

Least Squares Error (LSE): Sum of $(y - \hat{y})^2$

Hours (X)	Grade (Y) - Actual	Predicted ($\hat{Y}$) = $6.49 * (\text{Hours}) + 30.18 - 2.d.0$	Error ($Y - \hat{Y}$) – 2.d.p	Error ² – 2.d.p
6	82	69.12	12.88	165.89
10	88	95.08	-7.08	50.13
2	56	43.16	12.84	164.89
4	64	56.14	7.86	61.78
6	77	69.12	7.88	62.09
7	92	75.61	16.39	268.63
0	23	30.18	-7.18	51.55
1	41	36.67	4.33	18.75
8	80	82.10	-2.10	4.41
5	59	62.63	-3.63	13.18
3	47	49.65	-2.65	7.02

Summing up all the errors²: 867.32 (2.d.p)

Task 2

I. Which one of the following will be the best regression model? Justify your answer.

Model 1: $\hat{Y} = 22.60x + 169.2$

X : Advertising (Million Euros)	Y : Sales (Million Euros)	$\hat{Y} = 22.60$ (Advertising) + 169.2	Error (Y – $\hat{Y}$)	Error ²
23	651	689.0	-38.0	1444.0
26	762	756.8	5.2	27.0
30	856	847.2	8.8	77.4
34	1063	937.6	125.4	15725.2
43	1190	1141.0	49.0	2401.0
48	1298	1254.0	44.0	1936.0
52	1421	1344.4	76.6	5867.6
57	1440	1457.4	-17.4	302.8
58	1518	1480.0	38.0	1444.0

Summing up all the errors²: 29,224

Model 2: $\hat{Y} = 23.42x + 167.7$

X : Advertising (Million Euros)	Y : Sales (Million Euros)	$\hat{Y} = 23.42$ (Advertising) + 167.7	Error (Y – $\hat{Y}$)	Error ²
23	651	706.4	-55.4	3069.2
26	762	776.7	-14.7	216.1
30	856	870.3	-14.3	204.5
34	1063	964.0	99.0	9801.0
43	1190	1174.8	15.2	231.0
48	1298	1291.9	6.1	37.2
52	1421	1385.6	35.4	1253.2
57	1440	1502.9	-62.0	3956.4
58	1518	1526.3	-8.3	68.9

Summing up all the errors²: 18,837

Model 3: $\hat{Y} = 23.10x + 168.1$

X : Advertising (Million Euros)	Y : Sales (Million Euros)	$\hat{Y} = 23.10$ (Advertising) + 168.1	Error (Y – $\hat{Y}$)	Error ²
23	651	699.4	-48.4	2342.6
26	762	768.7	-6.7	44.9

30	856	861.1	-5.1	26.0
34	1063	953.5	109.5	11980.3
43	1190	1161.4	28.6	818.0
48	1298	1276.9	21.1	445.2
52	1421	1369.3	51.7	2673.0
57	1440	1484.8	-44.8	2007.0
58	1518	1507.9	10.1	102.0

Summing up all the errors²: 20,439

Best Least Square Error: $Y = 23.42x + 167.7$

Explanation: It had the smallest and therefore provided the best estimate of sales based on the advertising spending.

Task 3

- I. Model the Linear relationship between the marketing budget and sales.

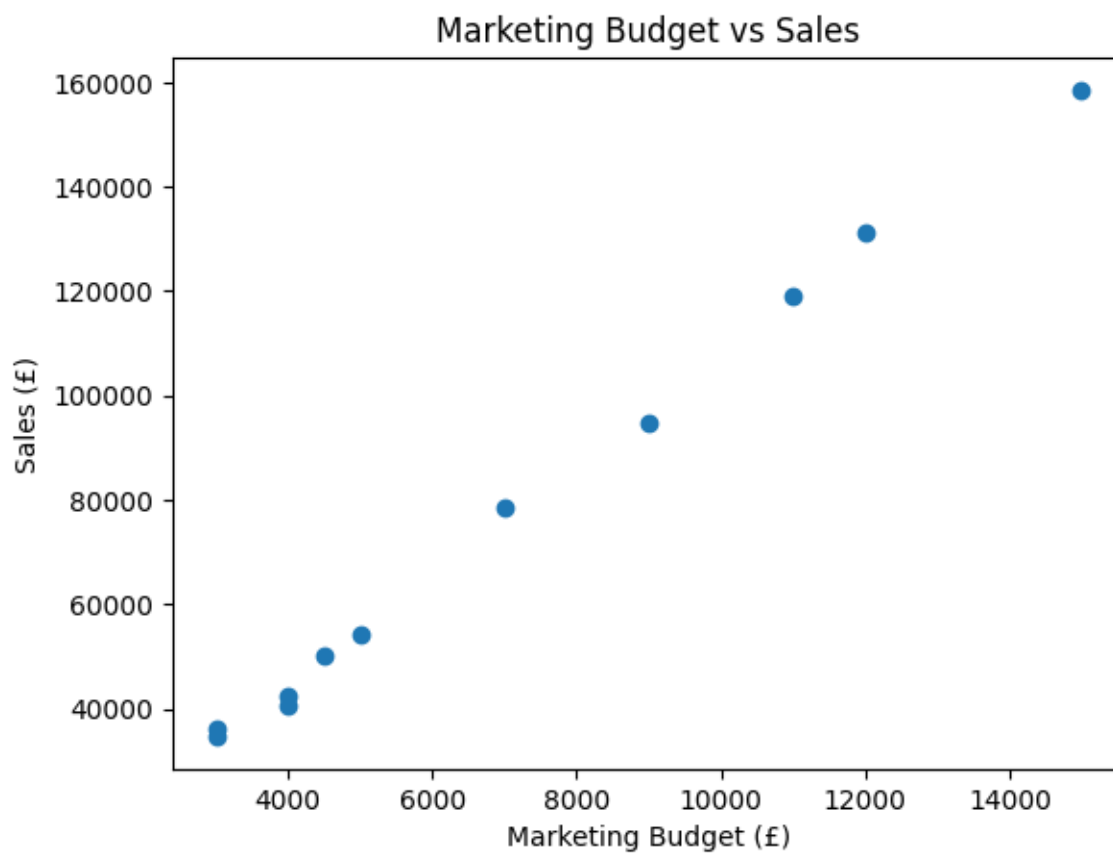


Figure 21: Marketing Budget vs Sales (Advertising Dataset)

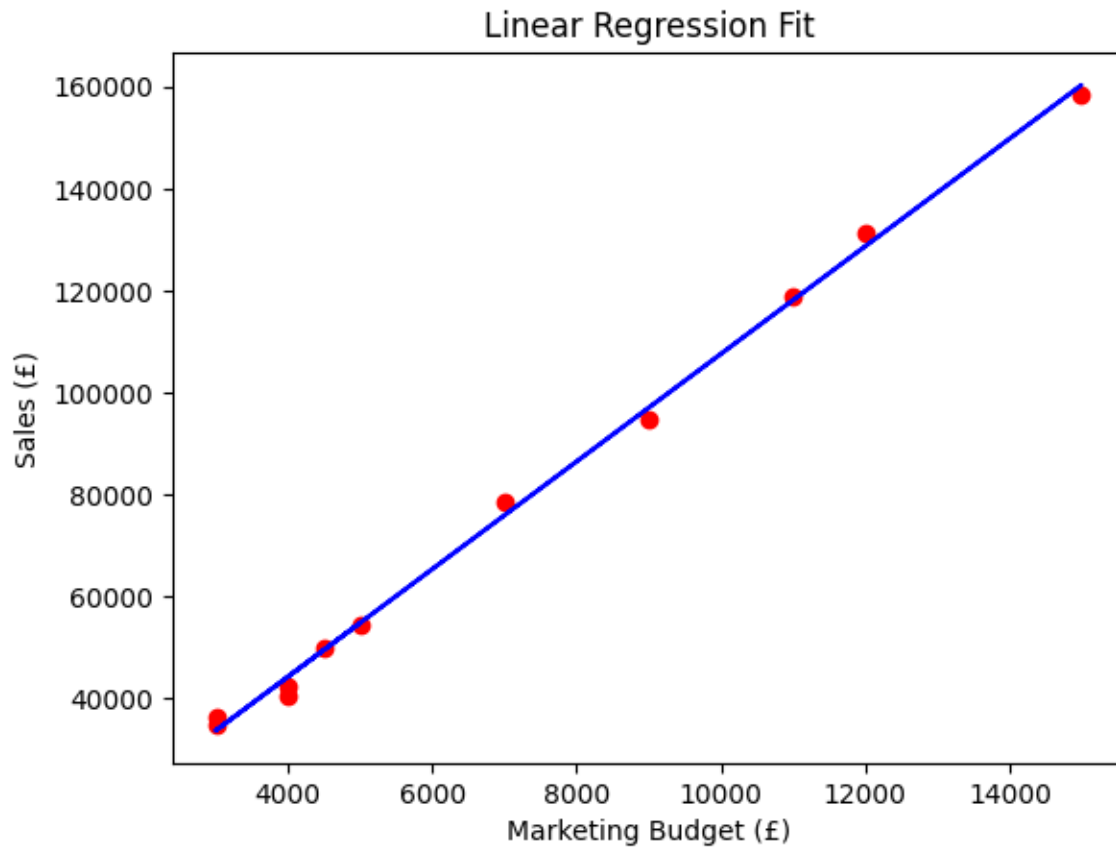


Figure 22: Linear Regression Fit for Marketing Budget vs Sales

- II. How much sales can we expect when the marketing budget is zero?

Linear regression equation: $Y = 10.55X + 2100.08$

$$Y = 10.55(0) + 2100.08$$

$$Y = 0 + 2100.08$$

$$Y = 2100.08$$

Expected sales at budget £0.00 = £2100.08

- III. For a marketing expenditure of £7,500, how much sales can we expect?

$$Y = 10.55(7,500.00) + 2100.08$$

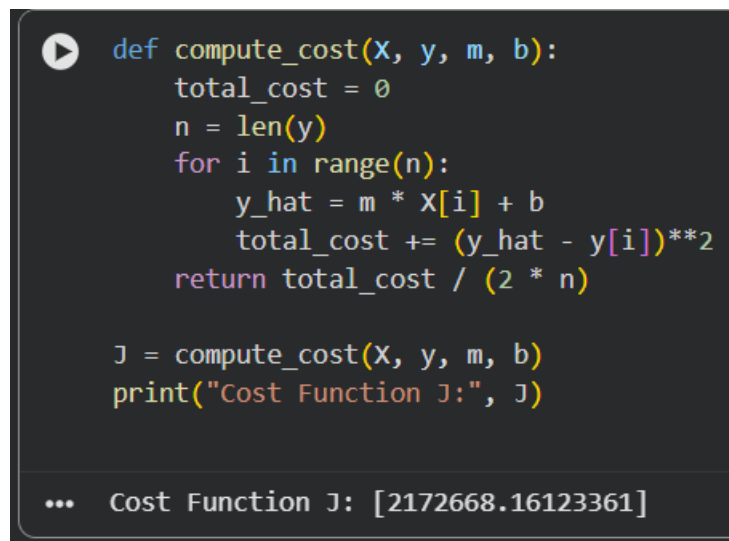
$$Y = 79,125.00 + 2100.08$$

$Y = 81,206.12$

Expected Sales at £7,500.00: £81,206.12

Task 4

- I. Modify the python script to compute the cost function J for the flitted line in Task 3. Provide a snapshot of your additions to the script. Also provide the results



```
def compute_cost(X, y, m, b):  
    total_cost = 0  
    n = len(y)  
    for i in range(n):  
        y_hat = m * X[i] + b  
        total_cost += (y_hat - y[i])**2  
    return total_cost / (2 * n)  
  
J = compute_cost(X, y, m, b)  
print("Cost Function J:", J)  
  
... Cost Function J: [2172668.16123361]
```

Result:

$J = 2,172,668.16$

This value represents the average squared error between the predicted sales and the actual sales values.

Although the Least Squares Error provides a measure of model fit, it does not verify whether the assumptions of linear regression are satisfied. Linear regression assumes a linear relationship between variables, independence of errors, constant variance of residuals and normally distributed errors. Without validating these assumptions using residual plots, the reliability of the model cannot be fully confirmed. Additionally, linear regression is limited to modelling linear relationships and is sensitive to outliers, which can disproportionately affect the fitted line and cost function value. For non-linear patterns, polynomial regression would be more appropriate, while Ridge or Lasso regression can address multicollinearity through regularization. For complex relationships involving feature interactions, models such as Random Forests or Neural Networks would likely provide superior predictive performance.

References

Zolgharni, M. (2025) *Machine Learning: Google Collab Notebooks (Weeks 3–11)*. School of Computing and Engineering, University of West London.

Zolgharni, M. (2025) *Machine Learning: Lecture slides (Weeks 1–11)*. School of Computing and Engineering, University of West London.

Kundu R. (2022) *F1 Score in Machine Learning: Intro & Calculation*. V7labs URL: <https://www.v7labs.com/blog/f1-score-guide> [Last Accessed: 05/11/2025]

Géron, A. (2022) *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3rd edn. O'Reilly Media. URL: https://shashwatwork.github.io/assets/files/ml_ebook.pdf [Last Accessed: 17/12/2025]

GeeksforGeeks (2025) *Curse of Dimensionality in Machine Learning*. URL: <https://www.geeksforgeeks.org/machine-learning/curse-of-dimensionality-in-machine-learning/> [Last Accessed: 01/01/2026]

GeeksforGeeks (2025) *What are the Advantages and Disadvantages of Random Forest?* URL: <https://www.geeksforgeeks.org/machine-learning/what-are-the-advantages-and-disadvantages-of-random-forest/> [Last Accessed: 02/01/2026].

Mikulasi, M. (2025) *What Is a Linear Kernel? (And Why It's Simpler Than You Think)* URL: <https://medium.com/@michalmikuli/what-is-a-linear-kernel-and-why-its-simpler-than-you-think-f2328991cca9> (Last Accessed: 03/01/2026]